Linux UART
开发指南

版本号: 2.2

发布日期: 2025.11.17

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2022.08.04	XAA0249	初始版本
1.1	2022.11.03	AWA1979	更新文档模板
1.2	2024.1.30	XAA0249	补充波特率配置说明
1.3	2024.3.14	XAA0312	更新文档内容
1.4	2024.12.24	XAA0249	补充两种驱动版本的使用说明
1.5	2025.2.17	XAA0249	补充软件 485 配置在不同版本驱动上的使用说明
1.6	2025.3.10	XAA0249	补充如何支持特性波特率
1.7	2025.4.8	XAA0249	修复格式问题
1.8	2025.4.7	XAA0249	补充控制台修改波特率问题
1.9	2025.7.21	XAA0249	补充修改波特率问题
2.0	2025.9.2	XAA0249	补充修改控制台等问题，优化文档格式与拼写错误
2.1	2025.11.12	XAA0249	修改数值单位约定等章节格式问题
2.2	2025.11.17	XAA0249	修改术语等格式问题

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 **1**
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1
1.4 文档约定 1
1.4.1 标志说明 1
1.4.2 地址与数据描述方法约定 2
1.4.3 数值单位约定 2
1.5 相关术语介绍 3
1.5.1 硬件术语 3
1.5.2 软件驱动版本介绍 3

2 模块介绍 **4**
2.1 模块功能介绍 4
2.2 模块流程介绍 4
2.3 驱动框架介绍 6
2.4 源码结构介绍 6
2.4.1 AW_UART 6
2.4.2 AW_UART_NG 6
2.5 模块配置介绍 7
2.5.1 AW_UART 7
2.5.1.1 menuconfig 配置说明 7
2.5.1.2 设备树配置说明 10
2.5.2 AW_UART_NG 12
2.5.2.1 menuconfig 配置说明 12
2.5.2.2 设备树配置说明 15
2.6 波特率配置说明 17
2.6.1 确认目标波特率是否属于特殊波特率 17
2.6.2 普通波特率配置 18
2.6.3 特殊波特率配置方法-查表 18
2.6.3.1 查表 18
2.6.3.2 查表后修改 dtsi 节点 19
2.7 模块特殊功能配置方法 20
2.7.1 AW_UART 20
2.7.1.1 DMA 模式配置 20
2.7.1.2 485 模式配置 21
2.7.2 AW_UART_NG 22
2.7.2.1 DMA 模式配置 22

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.7.2.2 485 模式 22

3 模块接口说明 24

3.1 打开/关闭串口 24

3.2 读/写串口 24

3.3 设置串口属性 24

3.3.1 tcgetattr 28

3.3.2 tcsetattr 28

3.3.3 cfgetispeed 29

3.3.4 cfgetospeed 30

3.3.5 cfsetispeed 30

3.3.6 cfsetospeed 30

3.3.7 cfsetspeed 30

3.3.8 tcflush 31

4 模块使用范例 32

5 调试方法 37

5.1 调试工具 37

5.2 调试节点 37

5.2.1 /sys/devices/platform/soc*/*uart*/dev_info 37

5.2.2 /sys/devices/platform/soc*/*uart*/status 38

5.2.3 /sys/devices/platform/soc*/*uart*/ctrl_info 38

6 常见需求及解决方案 39

6.1 控制台相关问题 39

6.1.1 如何切换控制台 39

6.1.2 如何切换控制台波特率 40

6.1.3 部分平台控制台 1.5 MHz 波特率相关问题 40

6.1.3.1 原因 40

6.1.3.2 硬件要求 41

6.1.3.3 如何切换回 115200 bps 波特率 42

6.2 关于 `uart` 波特率 42

6.2.1 特殊波特率配置方法-手动计算 42

6.2.1.1 获取支持特定波特率的时钟信息 42

6.2.1.2 计算后修改 `dtsi` 中的时钟配置 44

6.2.2 计算误码率 45

6.2.3 如何确认波特率设置成功 45

6.2.4 如何确认修改完 dtsi 后时钟切换成功 46

6.3 如何测试 UART 回环功能 46

6.4 接收数据时如何避免分段 47

版权所有 © 珠海全志科技股份有限公司。保留一切权利

插　图

图 2-1　初始化流程 5
图 2-2　内核 menuconfig 根菜单 7
图 2-3　Allwinner BSP 菜单 8
图 2-4　Device Drivers 菜单 9
图 2-5　UART Drivers 配置菜单 10
图 2-6　menuconfig 根目录 13
图 2-7　Allwinner bsp 下目录 13
图 2-8　Device Driver 目录 14
图 2-9　UART Driver 目录 15
图 2-10　image-20250804134918171 18
图 2-11　image-20250804134847665 19
图 2-12　image-20250804135112459 19
图 2-13　image-20250804140750379 20
图 2-14　image-20250804140956010 20
图 6-1　数据错位示例 41
图 6-2　接收数据分段 47

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 文档简介

介绍Linux内核中UART驱动的接口及使用方法，为 UART 设备的使用者提供参考。

1.2 目标读者

UART 驱动及应用层的开发/维护人员。

1.3 适用范围

表 1-1: 适用产品列表

内核版本	驱动文件
Linux-5.4	sunxi-uart.c 或 sunxi-uart-ng-core.c
Linux-5.10	sunxi-uart.c 或 sunxi-uart-ng-core.c
Linux-5.15	sunxi-uart.c 或 sunxi-uart-ng-core.c
Linux-6.1	sunxi-uart.c 或 sunxi-uart-ng-core.c
Linux-6.6	sunxi-uart.c 或 sunxi-uart-ng-core.c

1.4 文档约定

1.4.1 标志说明

⚠ 警告

- 提醒操作中应注意的事项。不当的操作可能会损坏器件，影响可靠性、降低性能等。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

注意

为准确理解文中指令、正确实施操作而提供的补充或强调信息。

技巧

一些容易忽视的小功能、技巧。了解这些功能或技巧能帮助解决特定问题或者节省操作时间。

1.4.2 地址与数据描述方法约定

本文档在描述地址、数据时遵循如下约定：

表 1-2: 地址与数据描述方法约定

符号	例子	说明
0x	0x0200，0x79	地址或数据以 16 进制表示。
0b	0b010，0b00 000 111	数据采用二进制表示 (寄存器描述除外)。
X	00X，XX1	数据描述中，X 代表 0 或 1。

1.4.3 数值单位约定

本文档在描述数据容量（如 NAND 容量）时，单位词头代表的是 1024 的倍数；描述频率、数据速率等时则代表的是 1000 的倍数。具体如下：

表 1-3: 数值单位约定

类型	符号	对应数值
数据容量（如 NAND 容量）	1 K	1024
数据容量（如 NAND 容量）	1 M	1 048 576
数据容量（如 NAND 容量）	1 G	1 073 741 824
频率，数据速率等	1 k	1000
频率，数据速率等	1 M	1 000 000
频率，数据速率等	1 G	1 000 000 000

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1.5 相关术语介绍

1.5.1 硬件术语

表 1-4: 硬件术语

术语	解释说明
sunxi	指 Allwinner 的一系列 SOC 硬件平台
UART	Universal Asynchronous Receiver/Transmitter，通用异步收发传输器
Console	控制台，Linux 内核中用于输出调试信息的 TTY 设备
TTY	TeleType/TeleTypewriters 的一个老缩写，原来指的是电传打字机，现在泛指和计算机串行端口连接的终端设备。TTY 设备还包括虚拟控制台，串口以及伪终端设备
FIFO	硬件缓冲区，用于暂存串口收发数据
DMA	Direct Memory Access(直接内存存取)

1.5.2 软件驱动版本介绍

在 SDK 中根据 IC 设计不同有两种驱动版本，可以将其称为`AW_UART`与`AW_UART_NG`，在代码发布时已经做好配置，不需要用户自行修改所用的软件驱动版本。

用户想使用特殊功能时：

- 需要先确认自己使用的是哪一种驱动版本，详细区分方法见章节 “menuconfig 配置说明” 或 “设备树配置说明”，两种方法均可区分驱动版本。
- 再按照 “模块特殊功能配置方法” 中对应章节的描述进行配置即可。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 模块介绍

2.1 模块功能介绍

通用异步收发器（Universal Asynchronous Receiver/Transmitter），通常称作UART，是一种串行、异步、全双工的通信协议，在嵌入式领域应用非常广泛。在 19 世纪 60 年代，为了解决计算机和电传打字机通信，Bell 发明了UART协议，将并行输入信号转换成串行输出信号。因为 UART 简单实用的特性，其已经成为一种使用非常广泛的通讯协议。全志的 uart 控制器支持以下功能：

- 兼容标准16550串口。
- 支持发送接收FIFO缓冲区（不同平台 FIFO 缓冲区大小有差异，具体情况见 user manual/datasheet）。
- 支持DMA控制接口。
- 支持软件/硬件流控配置。
- 支持2/4/8线串口接口。
- 支持5-8数据位和1/1.5/2停止位。
- 支持奇偶校验，无校验位。

2.2 模块流程介绍

下图为 UART 模块初始化流程介绍：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

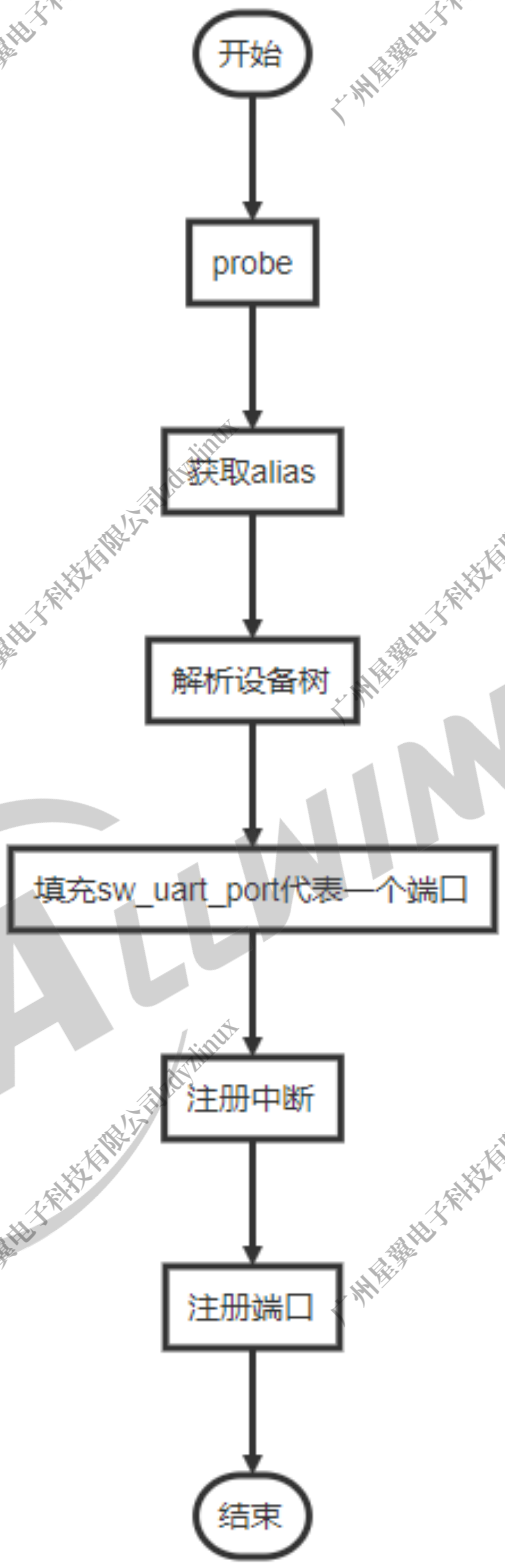

图 2-1: 初始化流程

广州星翼电子科技有限公司zdyzlinux

2.3 驱动框架介绍

Linux 内核中，UART 驱动的结构可以分为三个层次:

```
TTY core
   |
UART Core
   |
Sunxi UART Driver
```

1. `Sunxi UART Driver`，负责 SUNXI 平台 UART 控制器的初始化、数据通信等，也是我们要实现的部分。
2. `UART Core`，为 UART 驱动提供了一套 API, 完成设备和驱动的注册等。
3. `TTY core`，实现了内核中所有 TTY 设备的注册和管理。

2.4 源码结构介绍

2.4.1 AW_UART

```
|-- bsp
|     |-- drivers
|     |     |-- uart
|     |           |-- early_printk.c
|     |           |-- Kconfig
|     |           |-- Makefile
|     |           |-- sunxi-uart.c
|     |           |-- sunxi-uart.h
```

- `early_printk.c`：`earlyprintk`功能，一般打开。
- `sunxi-uart.c`：`sunxi`平台驱动核心文件。
- `sunxi-uart.h`：`sunxi`头文件，用来描述硬件信息。

2.4.2 AW_UART_NG

```
|-- bsp
|     |-- drivers
|     |     |-- uart
|     |           |-- early_printk.c
|     |           |-- Kconfig
|     |           |-- Makefile
|     |           |-- sunxi-uart-ng-core.c
|     |           |-- sunxi-uart-ng-DMA.c
|     |           |-- sunxi-uart-ng.h
```

- `early_printk.c`：`earlyprintk`功能，一般打开。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- sunxi-uart-ng-core.c：sunxi平台驱动核心文件。
- sunxi-uart-ng-dma.c：sunxi平台驱动 DMA 部分核心文件。
- sunxi-uart-ng.h：sunxi头文件，用来描述硬件信息。

2.5 模块配置介绍

2.5.1 AW_UART

2.5.1.1 menuconfig 配置说明

在 SDK 顶层目录，执行./build.sh menuconfig(需要先执行./build.sh config) 进入配置主界面，并按以下步骤操作：

- 首先，选择 Allwinner BSP 选项进入下一级配置，如下图所示。

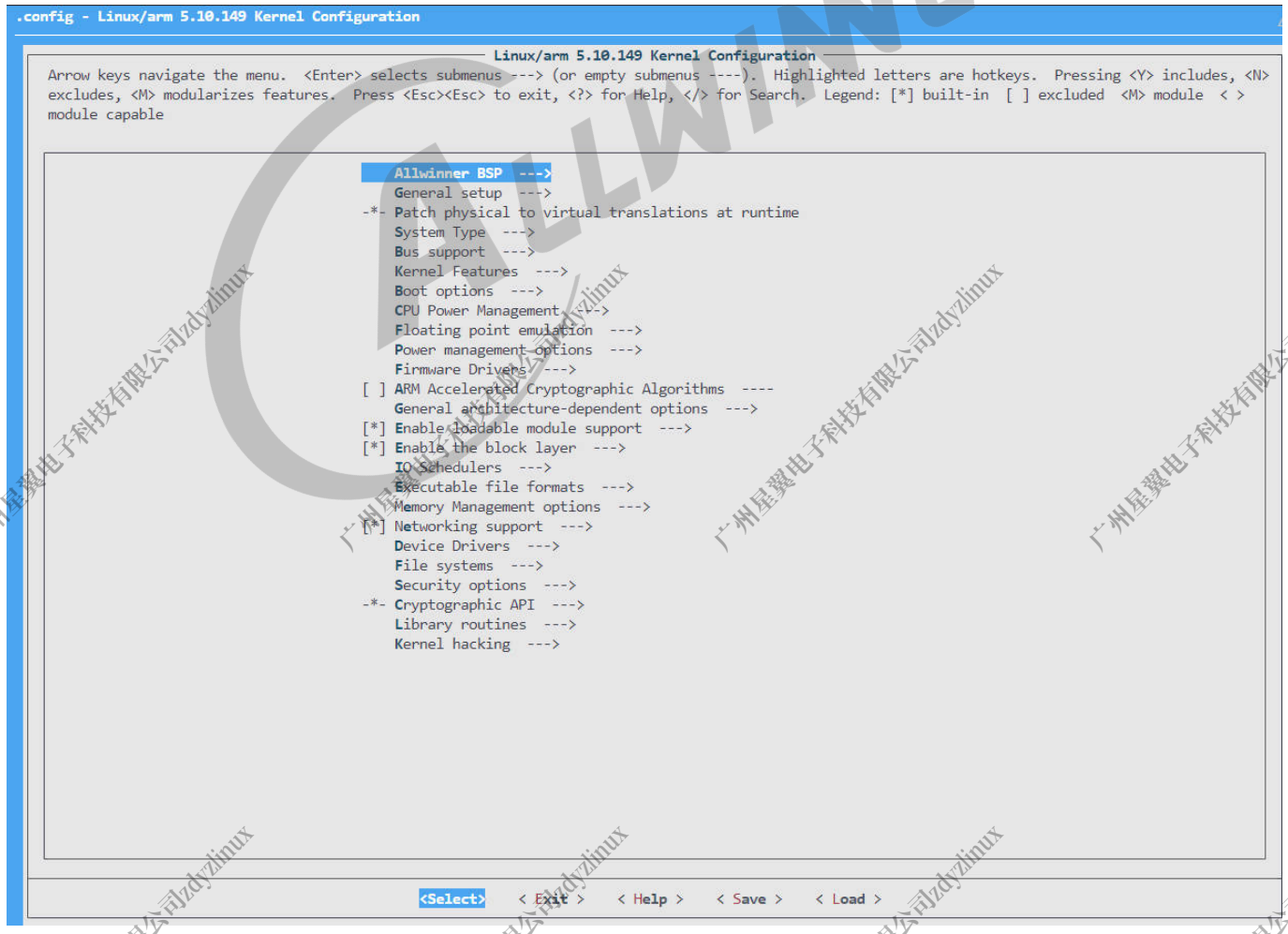

图 2-2: 内核 menuconfig 根菜单

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 选择 Device Drivers 进入下级配置，如下图所示：

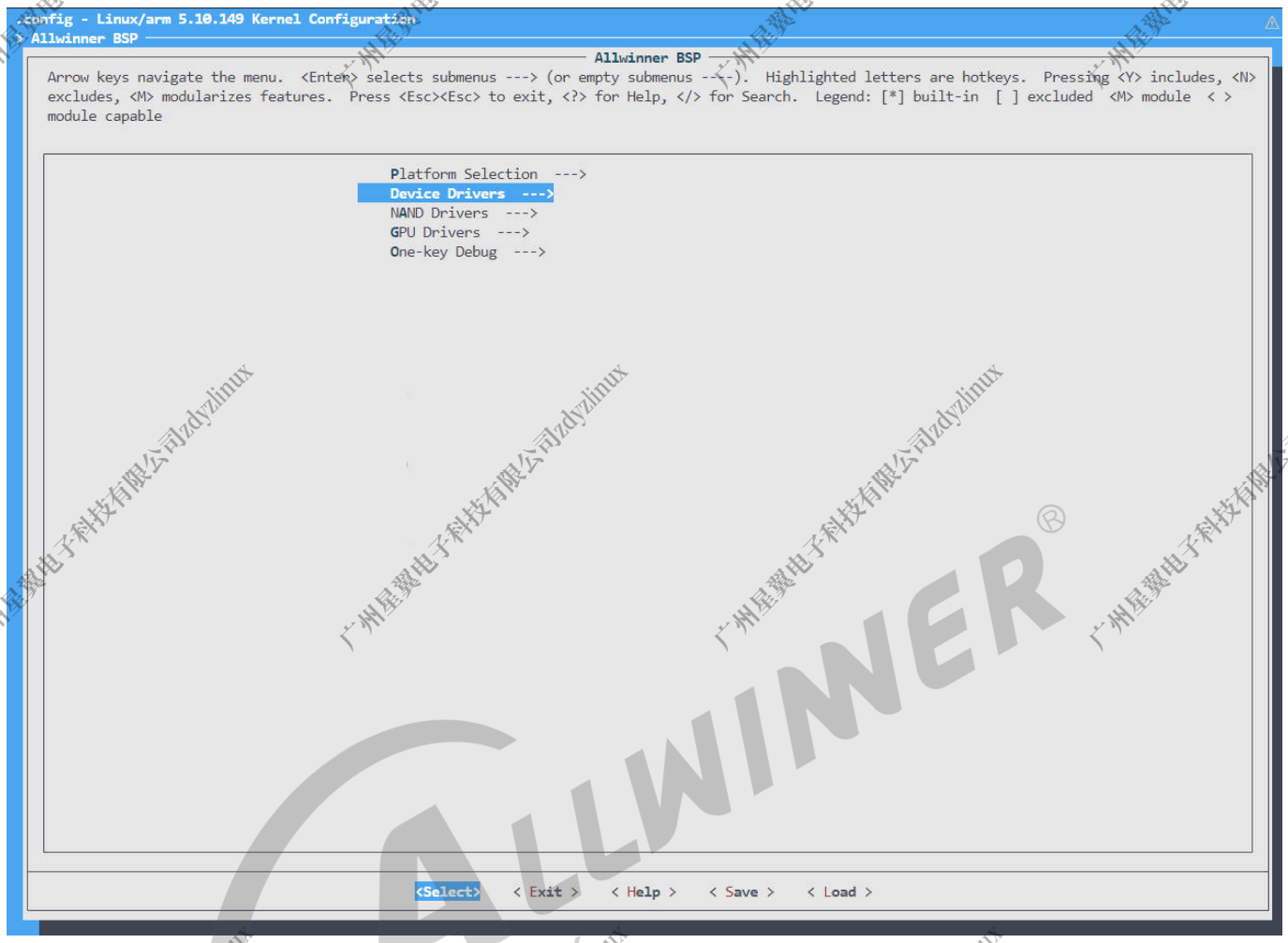

图 2-3: Allwinner BSP 菜单

- 选择 UART Drivers 进入下级配置，如下图所示：

广州星翼电子科技有限公司zdylinux

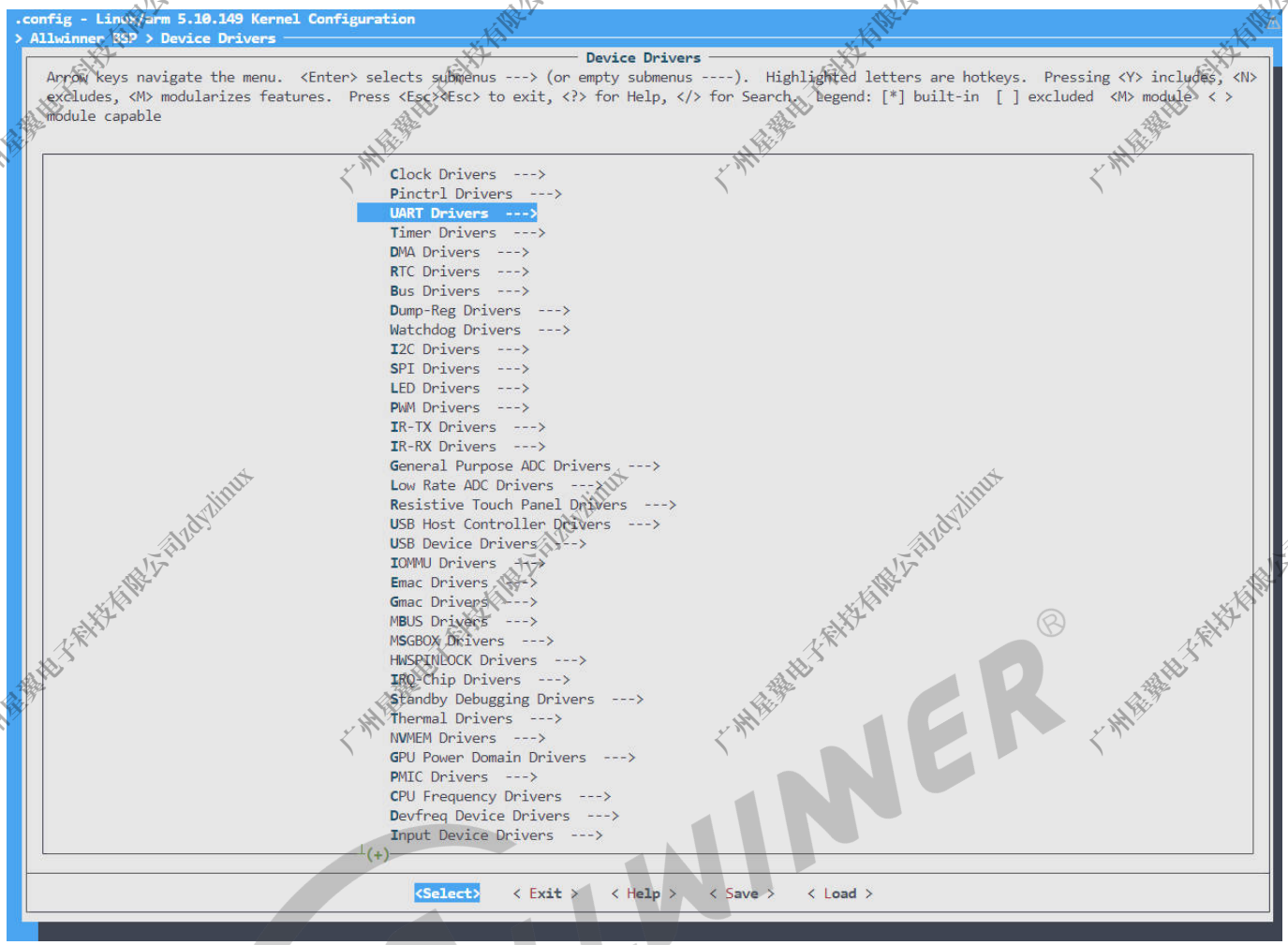

图 2-4: Device Drivers 菜单

- 选择 UART Support for Allwinner SoCs 和 Enable Console on UART 选项，如下图：
- 注意如果想使用 AW_UART 的 DMA 功能，需要打开图示中的`Enable DMA for UART`选项。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

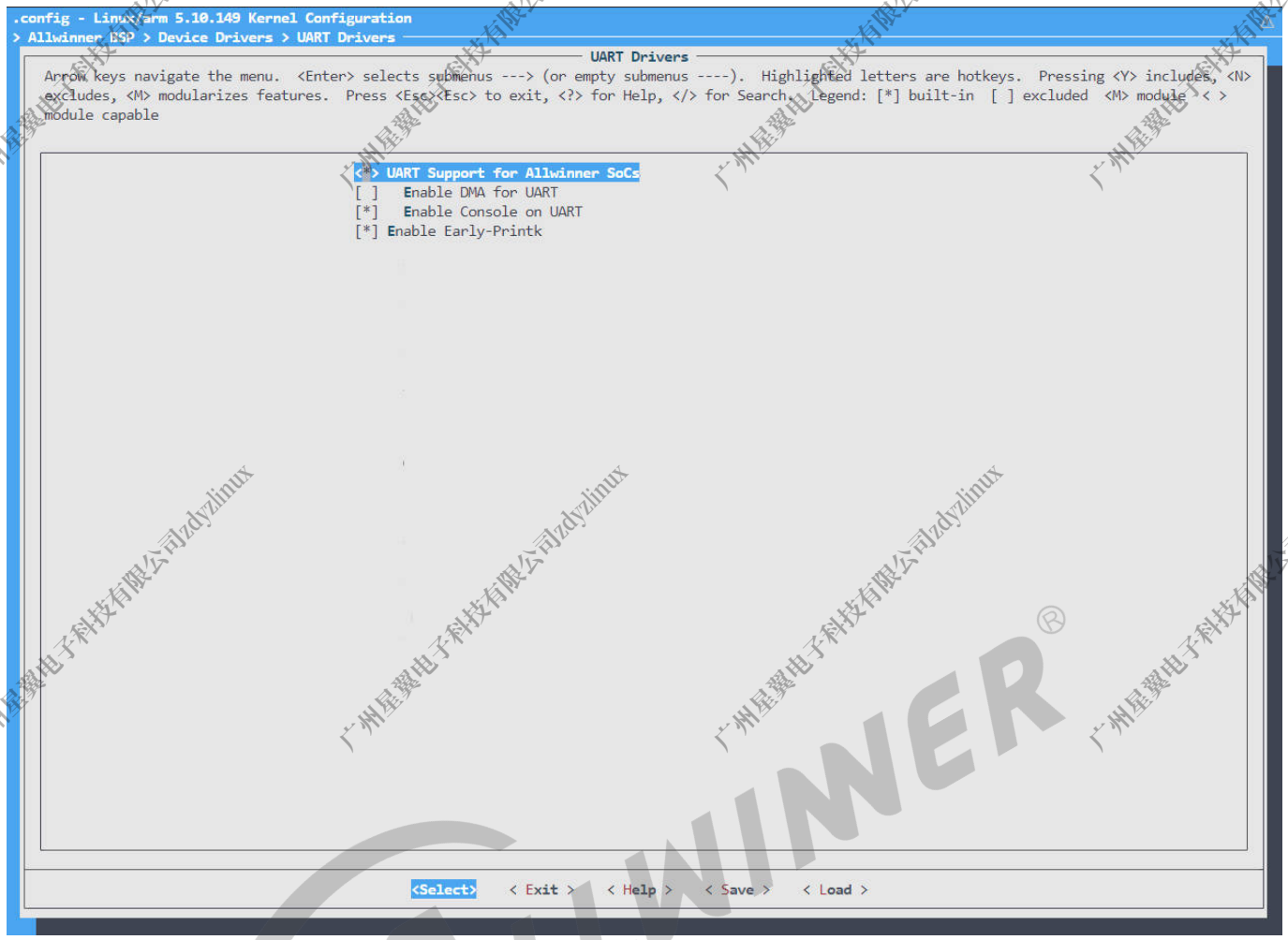

图 2-5: UART Drivers 配置菜单

2.5.1.2 设备树配置说明

- 设备树文件的配置是该SoC所有方案的通用配置，soc 设备树的路径为内核目录下：`bsp/configs/linux/sun*.dtsi`。
- 板级设备树路径：`device/config/chips/{IC}/configs/{BOARD}/linux-{kernel-ver}/board.dts`。

2.5.1.2.1 soc 设备树配置

```
uart0: uart@1c28000 {
    compatible = "allwinner,sun8i-uart";
    device_type = "uart0";
    reg = <0x0 0x01c28000 0x0 0x400>;
    interrupts = <GIC_SPI 1 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&ccu CLK_BUS_UART0>;          /* 设备使用的时钟 */
    resets = <&ccu RST_BUS_UART0>;          /* 设备reset时钟 */
    uart0_port = <0>;                       /* 端口号 */
    uart0_type = <2>;                       /* 该串口支持的模式，通常有2/4/8 */
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    sunxi,uart-fifosize = <64>;          /* FIFO缓冲区深度 */
    status = "disabled";
};
```

1. compatible表征具体的设备, 用于驱动和设备的绑定；（**注：客户无需关注**）

注意当驱动版本为 AW_UART 时，可选的 compatible 目前只有以下几种。

```
{ .compatible = "allwinner,sun8i-uart", },     { .compatible = "allwinner,sun50i-uart", },     {
.compatible = "allwinner,sun55i-uart", },     { .compatible = "allwinner,sun20i-uart", },     { .
compatible = "allwinner,sun251i-uart", },
```

2. reg设备使用的地址；（**注：客户无需关注**）
3. interrupts设备使用的中断；（**注：客户无需关注**）
4. clocks/reset设备使用的时钟；（**注：客户无需关注**）
5. uart0_port端口号；（**注：客户无需关注**）
6. uart0_type表示当前为几线模式；（**注：客户无需关注**）
7. sunxi,uart-fifosize表示FIFO缓冲区深度；（**注：客户无需关注**）
8. status是否使能该设备节点；（**注：客户无需关注**）

为了在UART 驱动代码中区分每一个UART 控制器，需要在Device Tree 中的 aliases 节点中为每一个UART 节点指定别名。别名形式为字符串 “serial” 加连续编号的数字，在 UART 驱动程序中可以通过of_alias_get_id() 函数获取对应的 UART 控制器的数字编号，从而区分每一个 UART 控制器。如下所示：

```
aliases {
    ......
    serial0 = &uart0;
    serial1 = &uart1;
    serial2 = &uart2;
    serial3 = &uart3;
    serial4 = &uart4;
    serial5 = &uart5;
    serial6 = &uart6;
    serial7 = &uart7;
    ......
};
```

2.5.1.2.2 板级设备树配置

board.dts 用于保存每个板级平台的设备信息 (如 demo 板、demo2.0 板等等)，board.dts 路径如下：

/device/config/chips/{IC}/configs/{BOARD}/board.dts。

在 board.dts 中的配置信息如果在 *.dtsi(如 sun50iw9p1.dtsi 等) 存在，则遵循以下规则：

1. 相同属性和结点，board.dts 的配置信息会覆盖 *.dtsi 中的配置信息。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2. 新增加的属性和结点，会添加到编译生成的 dtb 文件中。

board.dts配置示例如下：

```
&pio {
    uart0_pins_a: uart0_pins@0 {
        pins = "PB22", "PB23";
        function = "uart0";
    };

    uart0_pins_b: uart0_pins@1 {
        pins = "PB22", "PB23";
        function = "gpio_in";
    };
};

&uart0 {
    pinctrl-names = "default", "sleep";
    pinctrl-0 = <&uart0_pins_a>;
    pinctrl-1 = <&uart0_pins_b>;
    status = "okay";
};
```

1. “pinctrl-0”设备使用状态下的 GPIO 配置，“pinctrl-1”设备休眠状态下的 GPIO 配置；（**注：客户需关注，需与真正使用的 GPIO 引脚一致**）
2. status = "okay"当前uart的状态；（**注：客户需关注，当使用 uart 时需要打开**）
3. pins当前 uart 使用的硬件pin脚；（**注：若开启硬件流控时，需要添加rts及cts引脚，具体的引脚号需查看芯片手册**）

2.5.2 AW_UART_NG

2.5.2.1 menuconfig 配置说明

在 SDK 顶层目录，执行./build.sh menuconfig(需要先执行./build.sh config) 进入配置主界面，并按以下步骤操作：

- 首先，选择 Allwinner BSP 选项进入下一级配置，如下图所示。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
.config - Linux/riscv 6.6.0 Kernel Configuration
                    Linux/riscv 6.6.0 Kernel Configuration
  Arrow keys navigate the menu.  <Enter> selects submenus ---> (or empty submenus ----).  Highlighted letters are hotkeys.  Pressing <Y> includes, <N>
  excludes, <M> modularizes features.  Press <Esc><Esc> to exit, <?> for Help, </> for Search.  Legend: [*] built-in  [ ] excluded  <M> module  < > module
  capable

                    --> Allwinner BSP  --->
                        General setup  --->
                    -*- MMU-based Paged Memory Management Support
                        SoC selection  --->
                        CPU errata selection  --->
                        Platform type  --->
                        Kernel features  --->
                        Boot options  --->
                        Power management options  --->
                        CPU Power Management  --->
                    [ ] Virtualization  ----
                    [ ] ACPI (Advanced Configuration and Power Interface) Support  ----
                        General architecture-dependent options  --->
                    [*] Enable loadable module support  --->
                    [*] Enable the block layer  --->
                        Executable file formats  --->
                        Memory Management options  --->
                    [*] Networking support  --->
                        Device Drivers  --->
                        File systems  --->
                        Security options  --->
                    -*- Cryptographic API  --->
                        Library routines  --->
                        Kernel hacking  --->

                    <Select>    < Exit >    < Help >    < Save >    < Load >
```

图 2-6: menuconfig 根目录

- 选择 Device Drivers 进入下级配置，如下图所示：

```
.config - Linux/riscv 6.6.0 Kernel Configuration
> Allwinner BSP
                    Allwinner BSP
  Arrow keys navigate the menu.  <Enter> selects submenus ---> (or empty submenus ----).  Highlighted letters are hotkeys.  Pressing <Y> includes, <N>
  excludes, <M> modularizes features.  Press <Esc><Esc> to exit, <?> for Help, </> for Search.  Legend: [*] built-in  [ ] excluded  <M> module  < > module
  capable

                        Platform Information  --->
                        Allwinner's Debugging Options  --->
                        Allwinner's Turbo Options  --->
                    --> Device Drivers  --->
                        Library routines  --->
                        NAND Drivers  --->
                        GPU Drivers  --->

                    <Select>    < Exit >    < Help >    < Save >    < Load >
```

图 2-7: Allwinner bsp 下目录

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 选择 UART Drivers 进入下级配置，如下图所示：

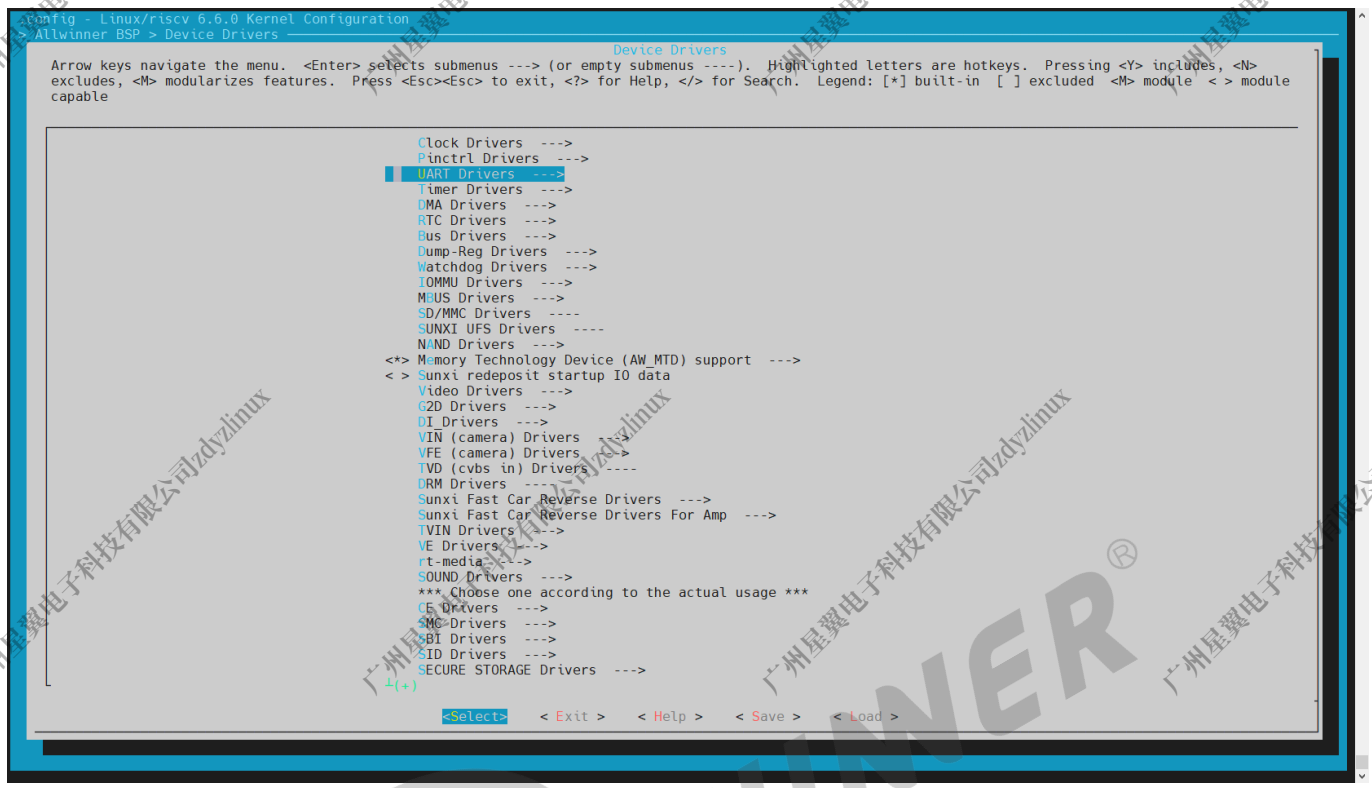

图 2-8: Device Driver 目录

- 选择 UART NG Support for Allwinner SoCs 和 Enable Console on UART 选项，如下图：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

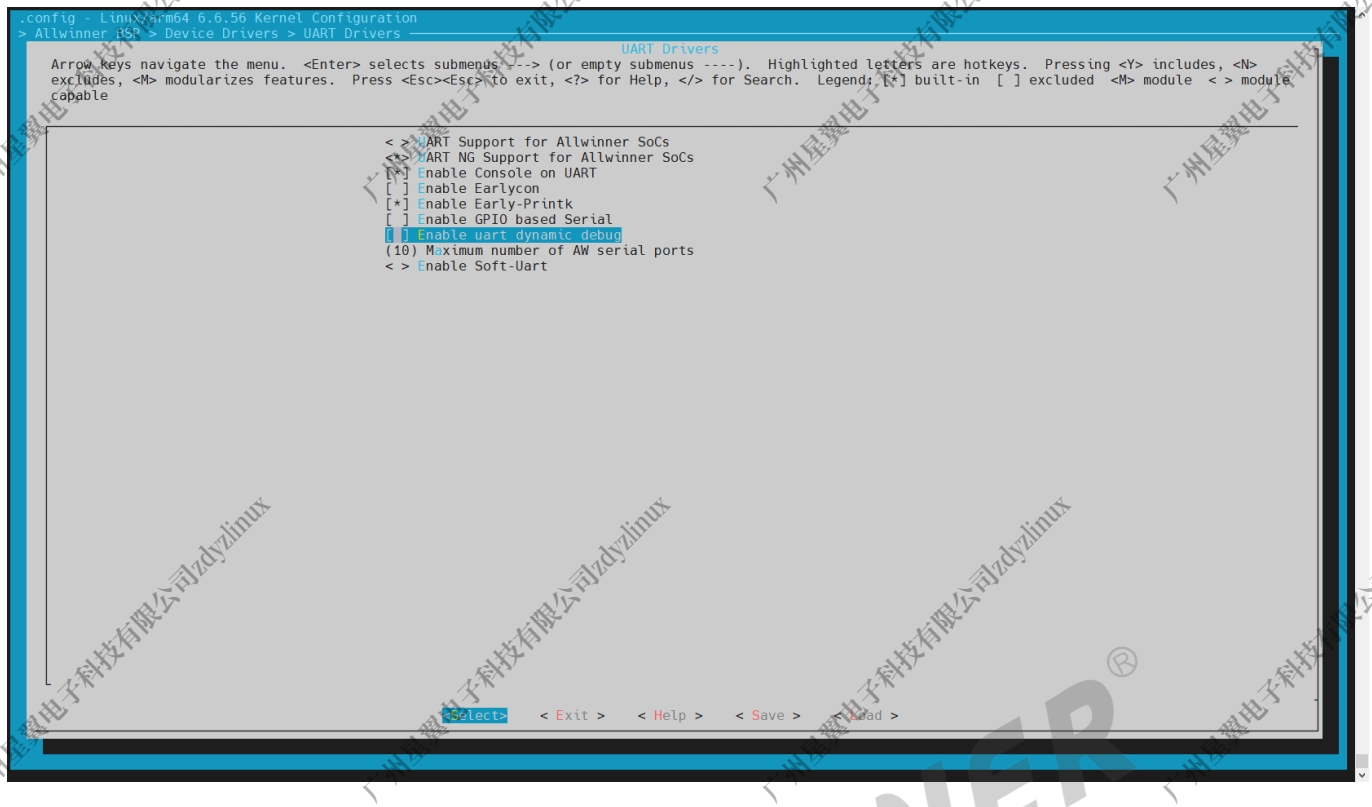

图 2-9: UART Driver 目录

2.5.2.2 设备树配置说明

- 设备树文件的配置是该SoC所有方案的通用配置，soc 设备树的路径为内核目录下：`bsp/configs/linux/sun*.dtsi`。
- 板级设备树路径：`device/config/chips/{IC}/configs/{BOARD}/linux-{kernel-ver}/board.dts`。

2.5.2.2.1 soc 设备树配置

```
uart0: uart@1c28000 {
    compatible = "allwinner,uart-v100";
    device_type = "uart0";
    reg = <0x0 0x01c28000 0x0 0x400>;
    interrupts = <GIC_SPI 1 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&ccu CLK_BUS_UART0>;        /* 设备使用的时钟 */
    resets = <&ccu RST_BUS_UART0>;        /* 设备reset时钟 */
    uart0_port = <0>;                     /* 端口号 */
    uart0_type = <2>;                     /* 该串口支持的模式，通常有2/4/8 */
    sunxi,uart-fifosize = <64>;           /* FIFO缓冲区深度 */
    status = "disabled";
};
```

1. `compatible`表征具体的设备，用于驱动和设备的绑定；（**注：客户无需关注**）

版权所有 © 珠海全志科技股份有限公司。保留一切权利

注意当驱动版本为 AW_UART 时，可选的 compatible 目前只有以下几种

```
{ .compatible = "allwinner,uart-v100",          .data = &sunxi_uart_v100_data},     { .compatible =
"allwinner,uart-v101",          .data = &sunxi_uart_v101_data},
```

2. reg设备使用的地址；（**注：客户无需关注**）
3. interrupts设备使用的中断；（**注：客户无需关注**）
4. clocks/reset设备使用的时钟；（**注：客户无需关注**）
5. uart0_port端口号；（**注：客户无需关注**）
6. uart0_type表示当前为几线模式；（**注：客户无需关注**）
7. sunxi,uart-fifosize表示FIFO缓冲区深度;（**注：客户无需关注**）
8. status是否使能该设备节点；（**注：客户无需关注**）

为了在UART 驱动代码中区分每一个UART 控制器，需要在Device Tree 中的 aliases 节点中为每一个UART 节点指定别名。别名形式为字符串 “serial” 加连续编号的数字，在 UART 驱动程序中可以通过of_alias_get_id() 函数获取对应的 UART 控制器的数字编号，从而区分每一个 UART 控制器。如下所示：

```
aliases {
    ......
    serial0 = &uart0;
    serial1 = &uart1;
    serial2 = &uart2;
    serial3 = &uart3;
    serial4 = &uart4;
    serial5 = &uart5;
    serial6 = &uart6;
    serial7 = &uart7;
    ......
};
```

2.5.2.2.2 板级设备树配置

board.dts 用于保存每个板级平台的设备信息 (如 demo 板、demo2.0 板等等)，board.dts 路径如下：

/device/config/chips/{IC}/configs/{BOARD}/board.dts。

在 board.dts 中的配置信息如果在 *.dtsi(如 sun50iw9p1.dtsi 等) 存在，则遵循以下规则：

1. 相同属性和结点，board.dts 的配置信息会覆盖 *.dtsi 中的配置信息。
2. 新增加的属性和结点，会添加到编译生成的 dtb 文件中。

board.dts配置示例如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
&pio {
    uart0_pins_a: uart0_pins@0 {
        pins = "PB22", "PB23";
        function = "uart0";
    };

    uart0_pins_b: uart0_pins@1 {
        pins = "PB22", "PB23";
        function = "gpio_in";
    };
};

&uart0 {
    pinctrl-names = "default", "sleep";
    pinctrl-0 = <&uart0_pins_a>;
    pinctrl-1 = <&uart0_pins_b>;
    status = "okay";
};
```

1. “pinctrl-0”设备使用状态下的 GPIO 配置，“pinctrl-1”设备休眠状态下的 GPIO 配置；（**注：客户需关注，需与真正使用的 GPIO 引脚一致**）
2. status = "okay"当前uart的状态；（**注：客户需关注，当使用 uart 时需要打开**）
3. pins当前 uart 使用的硬件pin脚；（**注：若开启硬件流控时，需要添加rts及cts引脚，具体的引脚号需查看芯片手册**）

2.6 波特率配置说明

波特率配置的一般流程为：

- 先确认想使用的波特率是否属于特殊波特率。
- 如果不属于，配置方法参考普通波特率配置章节。
- 如果属于，则参考特殊波特率配置方法-查表章节。

配置完成后如果想确认波特率配置是否正确，请查看如何确认波特率设置成功章节中的验证方法。

2.6.1 确认目标波特率是否属于特殊波特率

一般情况下，apb时钟频点为24 MHz，可直接支持的常用波特率有：4800 bps、9600 bps、19200 bps、57600 bps、115200 bps、1500000 bps 等。

如果波特率大于1.5M bps，则直接属于特殊波特率，如果波特率小于1.5M bps，则需要通过计算进一步判断。

计算1500000/目标波特率的结果。

- 若结果为整数，则当前可以直接支持。
- 若结果带小数，请参考计算误码率章节，若误码率过高（建议小于 1.725%），则属于特殊波特率。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.6.2 普通波特率配置

用户使用uart时，可以通过cfsetispeed、cfsetospeed等用户态接口设置某笔传输的波特率。

使用示例：

```
...
int fd;
int baudrate = 115200;
struct termios options;

fd = uart_open(name1);

if (tcgetattr(fd, &options) != 0) {
    DBG("tcgetattr failed\n");
    goto SET_PARAM_FAIL;
}

cfsetispeed(&options, baudrate);
cfsetospeed(&options, baudrate);

if (tcsetattr(fd, TCSANOW, &options) != 0) {
    DBG("tcsetattr failed\n");
    goto SET_PARAM_FAIL;
}
...
```

2.6.3 特殊波特率配置方法-查表

目标波特率为特殊波特率意味着需要修改 uart 上层时钟频点，有两种方法获取匹配的频点：查表和计算。

优先使用查表法，需要先找到平台对应的《XXXX 平台 uart 波特率查询表格》，若是没有平台对应表格，则需参考特殊波特率配置方法-手动计算章节，需手动计算时钟源，并修改dtsi文件。

2.6.3.1 查表

进入 “uart 波特率排序” 页，在 “uart 波特率” 列中查找目标波特率，此列已经按照从大到小的顺序排列。

- 如果能够直接搜到目标波特率，则按照 “查表后修改 dtsi 节点” 章节指导修改文件后重新编译。

示例如下，这种情况表示可以无误差的支持 3125000 波特率。

apb父时钟名称	apb父时钟频率(Hz)	apb分频值	apb频率	波特率分频值	uart波特率
PERI_600M_BUS	600000000	6	100000000	2	3125000

图 2-10: image-20250804134918171

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 如果不能直接搜到目标波特率，但是能找到目标波特率附近的波特率，且误码率较小（建议小于 1.725%），则按照 “查表后修改 dtsi 节点” 章节指导修改文件后重新编译，误码率计算方法见 “计算误码率” 章节。

示例如下，若目标波特率为 520000，则此种情况下误码率很小，也认为可以支持。

apb父时钟名称	apb父时钟频率(HZ)	apb分频值	apb频率	波特率分频值	uart波特率
PERI_600M_BUS	600000000	6	100000000	12	520833

图 2-11: image-20250804134847665

- 如果不能直接搜到目标波特率，且目标波特率附近的波特率误码率都较大，则表示此平台无法支持目标波特率。

示例如下，若目标波特率为 3000000，查表发现接近的波特率误码率很大，认为此平台不可支持 3000000 的波特率。

apb父时钟名称	apb父时钟频率(HZ)	apb分频值	apb频率	波特率分频值	uart波特率
PERI_600M_BUS	600000000	12	50000000	1	3125000
PERI_600M_BUS	600000000	13	46153846	1	2884615

图 2-12: image-20250804135112459

2.6.3.2 查表后修改 dtsi 节点

`dtsi`文件的路径为`SDK/bsp/configs/linux-XX/sunXXiwXX.dtsi`，以下代码为例，讲解 dtsi 中有关时钟的两个重要节点。

```
    uart2: uart@5000800 {
            ...
            device_type = "uart2";
            reg = <0x0 0x05000800 0x0 0x400>;
            interrupts = <GIC_SPI 2 IRQ_TYPE_LEVEL_HIGH>;
-           clocks = <&ccu CLK_BUS_UART2>;
+           clocks = <&ccu CLK_BUS_UART2>,
+               <&ccu CLK_APB_UART>,
+               <&ccu PERI_600M_BUS>;
+           clock-frequency = <100000000>;
            pinctrl-names = "default", "sleep";
            ...
    }
```

2.6.3.2.1 clocks 节点

当需要调整 uart 的时钟源频点，以支持特定波特率时，需要填写三个时钟信息，依次为

- uart 时钟宏名，示例代码中的`<&ccu CLK_BUS_UART2>`

  `uart`时钟宏名是必填项，将`dtsi`文件中原有的`clocks`节点部分继续放在新的`clocks`节点的第一个成员即可。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- apb 时钟宏名，示例代码中的`<&ccu CLK_APB_UART>`

在《XXXX 平台 uart 波特率查询表格》的 “配置参数页” 查询apb时钟宏名后填入dtsi文件的clocks节点第二个成员。

apb时钟宏名	CLK_APB_UART

图 2-13: image-20250804140750379

- apb 时钟源宏名，示例代码中的`<&ccu PERI_600M_BUS>`

在《XXXX 平台 uart 波特率查询表格》中，进入 “uart 波特率排序” 页，在 “uart 波特率” 列中查找到目标波特率后，将 “apb 时钟源宏名” 中的内容填入dtsi文件的clocks节点第三个成员。

apb时钟源宏名	apb父时钟频率(H	apb分频值	apb频率	波特率分频值	uart波特率
PERI_600M_BUS	600000000	6	100000000	1	6250000

图 2-14: image-20250804140956010

2.6.3.2.2 clock-frequency 节点

示例代码中的`clock-frequency = <100000000>;`。

在《XXXX 平台 uart 波特率查询表格》中，进入 “uart 波特率排序” 页，在 “uart 波特率” 列中查找到目标波特率后，将 “apb 频率” 中的内容填入dtsi文件的clock-frequency节点，当表格中有多组配置都可以支持到目标波特率时，选 “apb” 频率最大的一组。

- 重要注意事项！！！
  - 由于所有uart时钟都来自于同一个apb时钟，因此dtsi的修改需要同步到改到所有的uart节点中，不可以只修改其中一个，否则会影响其他uart口的正常数据传输，例如可能会导致控制台的打印出现乱码。
  - 修改APB总线时钟，可能会影响到使用到APB总线时钟下挂的其他模块，例如twi等，需要根据ccmu spec查询是否会影响到其他模块。

2.7 模块特殊功能配置方法

2.7.1 AW_UART

2.7.1.1 DMA 模式配置

使用前提：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

当使用CPU传输，出现系统负载过大、响应不及时、overrun 等问题时，可以使用DMA模式来降低CPU负载。

未出现上述异常情况，请使用默认的cpu传输模式，因为DMA模式不一定能够提升传输效率。

DMA模式配置流程如下：

1. 在内核配置菜单打开CONFIG_AW_SERIAL_DMA配置。
2. 在对应dts配置开启DMA，如下所示:

```
&uart6 {
    ......
    use_dma = <3>;  /* 是否采用DMA 方式传输，0：不启用，1：只启用TX，2：只启用RX，3：启用TX 与RX */
    dmas = <&dma 14>, <&dma 14>;    /* 14表示DRQ，查看DMA spec */
    dma-names = "tx", "rx";
    ......
};
```

2.7.1.2 485 模式配置

AW_UART 版本驱动只支持软件控制的 485 管脚，需要添加以下成员：

- sunxi,uart-rs485
  - 0：485 模式关闭。
  - 1：485 模式使能。
- sunxi,uart-485fl
  - 0：485 gpio 管脚电平为低时表示接收状态。
  - 1：485 gpio 管脚电平为高时表示接收状态。
- sunxi,uart-485oe-gpios
  - 用于外部转换芯片使能信号的 gpio 引脚，GPIO_ACTIVE_HIGH 含义为默认为高电平。

示例：

```
uart1: uart@2500400 {
            ...
            status = "okay";
            //添加以下变更
            sunxi,uart-rs485 = <1>;
            sunxi,uart-485fl = <1>;
            sunxi,uart-485oe-gpios = <&pio PG 8 GPIO_ACTIVE_HIGH>;
        };
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.7.2 AW_UART_NG

2.7.2.1 DMA 模式配置

使用前提：

当使用cpu传输，出现系统负载过大、响应不及时、overrun 等问题时，可以使用DMA模式来经降低CPU负载。

若传输数据量较小时，请使用默认的CPU传输模式，因为DMA模式不一定能够提升传输效率。

DMA模式配置流程如下：

1. DMA 传输模式下设备树节点需要增加以下配置。

```
&uart6 {
    ......
    dmas = <&dma 15>, <&dma 15>;
    dma-names = "tx", "rx"; /* 需要修改为tx/rx，和非DMA传输模式有差异 */
    status = "disabled";
    ......
};
```

2.7.2.2 485 模式

2.7.2.2.1 驱动版本号 1.1.3 及之前

需要添加以下成员：

- linux,rs485-enabled-at-boot-time

示例

```
uart1: uart@2500400 {
            ...
            status = "okay";
            //添加以下变更
            linux,rs485-enabled-at-boot-time;
        };
```

2.7.2.2.2 驱动版本号 1.1.4 及之后

需要添加以下成员：

- sunxi,uart-485pin_auto = <1>;（与 sunxi,uart-rs485 都需要配置，是为了兼容不同小版本的驱动代码）

版权所有 © 珠海全志科技股份有限公司。保留一切权利

  - 0：485 模式关闭。
  - 1：485 模式使能。

- sunxi,uart-rs485
  - 0：485 模式关闭。
  - 1：软件控制 485 传输方向模式使能。

- sunxi,uart-485fl
  - 0：485 gpio 管脚电平为低时表示接收状态。
  - 1：485 gpio 管脚电平为高时表示接收状态。

- sunxi,uart-485oe-gpios
  - 用于外部转换芯片使能信号的 gpio 引脚，GPIO_ACTIVE_HIGH 含义为默认为高电平。

示例：

```
uart1: uart@2500400 {
            ...
            status = "okay";
            //添加以下变更
            sunxi,uart-485pin_auto = <1>;
            sunxi,uart-rs485 = <1>;
            sunxi,uart-485fl = <1>;
            sunxi,uart-485oe-gpios = <&pio PG 8 GPIO_ACTIVE_HIGH>;
        };
```

在 dtsi 中需对 compatible 进行配置。

示例：

```
uart4: uart@2604000 {

      compatible = "allwinner,uart-v100";

      device_type = "uart4";

        .....

} ;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 模块接口说明

UART 驱动会注册生成串口设备/dev/ttySx，应用层的使用只需遵循Linux系统中的标准串口编程方法即可。

3.1 打开/关闭串口

使用标准的文件打开函数：

```
int open(const char *pathname, int flags);
int close(int fd);
```

需要引用头文件：

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
```

3.2 读/写串口

同样使用标准的文件读写函数：

```
ssize_t read(int fd, void *buf, size_t count);
ssize_t write(int fd, const void *buf, size_t count);
```

需要引用头文件：

```
#include <unistd.h>
```

3.3 设置串口属性

串口属性包括波特率、数据位、停止位、校验位、流控等，这部分是串口设备特有的接口。串口属性的数据结构 termios 定义如下：（terminos.h）。

```
#define NCCS 19
struct termios {
    tcflag_t c_iflag;       /* input mode flags */
    tcflag_t c_oflag;       /* output mode flags */
    tcflag_t c_cflag;       /* control mode flags */
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    tcflag_t c_lflag;        /* local mode flags */
    cc_t c_line;             /* line discipline */
    cc_t c_cc[NCCS];         /* control characters */
};
```

其中，c_iflag 的标志常量定义如下：

标志	说明
IGNBRK	忽略输入中的 BREAK 状态。
BRKINT	如果设置了 IGNBRK，将忽略 BREAK。如果没有设置，但是设置了 BRKINT，那么 BREAK 将使得输入和输出队列被刷新，如果终端是一个前台进程组的控制终端，这个进程组中所有进程将收到 SIGINT 信号。如果既未设置 IGNBRK 也未设置 BRKINT，BREAK 将视为与 NUL 字符同义，除非设置了 PARMRK，这种情况下它被视为序列 \377 \0 \0。
IGNPAR	忽略桢错误和奇偶校验错。
PARMRK	如果没有设置 IGNPAR，在有奇偶校验错或桢错误的字符前插入 \377 \0。如果既没有设置 IGNPAR 也没有设置 PARMRK，将有奇偶校验错或桢错误的字符视为 \0。
INPCK	启用输入奇偶检测。
ISTRIP	去掉第八位。
INLCR	将输入中的 NL 翻译为 CR。
IGNCR	忽略输入中的回车。
ICRNL	将输入中的回车翻译为新行 (除非设置了 IGNCR)。
IUCLC	(不属于 POSIX) 将输入中的大写字母映射为小写字母。
IXON	启用输出的 XON/XOFF 流控制。
IXANY	(不属于 POSIX.1；XSI) 允许任何字符来重新开始输出。
IXOFF	启用输入的 XON/XOFF 流控制。
IMAXBEL	(不属于 POSIX) 当输入队列满时响零。Linux 没有实现这一位，总是将它视为已设置。

c_oflag 的标志常量定义如下：

标志	说明
OLCUC	(不属于 POSIX) 将输出中的小写字母映射为大写字母。
ONLCR	(XSI) 将输出中的新行符映射为回车-换行。
OCRNL	将输出中的回车映射为新行符。
ONOCR	不在第 0 列输出回车。
ONLRET	不输出回车。
OFILL	发送填充字符作为延时，而不是使用定时来延时。
OFDEL	(不属于 POSIX) 填充字符是 ASCII DEL (0177)。如果不设置，填充字符则是 ASCII NUL。
NLDLY	新行延时掩码。取值为 NL0 和 NL1。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

标志	说明
CRDLY	回车延时掩码。取值为 CR0, CR1, CR2, 或 CR3。
TABDLY	水平跳格延时掩码。取值为 TAB0, TAB1, TAB2, TAB3 (或 XTABS)。取值为 TAB3，即 XTABS，将扩展跳格为空格 (每个跳格符填充 8 个空格)。
BSDLY	回退延时掩码。取值为 BS0 或 BS1。(从来没有被实现过)。
VTDLY	竖直跳格延时掩码。取值为 VT0 或 VT1。
FFDLY	进表延时掩码。取值为 FF0 或 FF1。

c_cflag 的标志常量定义如下：

标志	说明
CBAUD	(不属于 POSIX) 波特率掩码 (4+1 位)。
CBAUDEX	(不属于 POSIX) 扩展的波特率掩码 (1 位)，包含在 CBAUD 中。(POSIX 规定波特率存储在 termios 结构中，并未精确指定它的位置，而是提供了函数 cfgetispeed() 和 cfsetispeed() 来存取它。一些系统使用 c_cflag 中 CBAUD 选择的位，其他系统使用单独的变量，例如 sg _ispeed 和 sg_ospeed 。)
CSIZE	字符长度掩码。取值为 CS5, CS6, CS7, 或 CS8。
CSTOPB	设置两个停止位，而不是一个。
CREAD	打开接受者。
PARENB	允许输出产生奇偶信息以及输入的奇偶校验。
PARODD	输入和输出是奇校验。
HUPCL	在最后一个进程关闭设备后，降低 modem 控制线 (挂断)。
CLOCAL	忽略 modem 控制线。
LOBLK	(不属于 POSIX) 从非当前 shell 层阻塞输出 (用于 shl)。
CIBAUD	(不属于 POSIX) 输入速度的掩码。CIBAUD 各位的值与 CBAUD 各位相同，左移了 IBSHIFT 位。
CRTSCTS	(不属于 POSIX) 启用 RTS/CTS (硬件) 流控制。

c_lflag 的标志常量定义如下：

标志	说明
ISIG	当接受到字符 INTR, QUIT, SUSP, 或 DSUSP 时，产生相应的信号。
ICANON	启用标准模式 (canonical mode)。允许使用特殊字符 EOF, EOL, EOL2, ERASE, KILL, LNEXT, REPRINT, STATUS, 和 WERASE，以及按行的缓冲。
XCASE	(不属于 POSIX; Linux 下不被支持) 如果同时设置了 ICANON，终端只有大写。输入被转换为小写，除了以 前缀的字符。输出时，大写字符被前缀，小写字符被转换成大写。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

标志	说明
ECHO	回显输入字符。
ECHOE	如果同时设置了 ICANON，字符 ERASE 擦除前一个输入字符，WERASE 擦除前一个词。
ECHOK	如果同时设置了 ICANON，字符 KILL 删除当前行。
ECHONL	如果同时设置了 ICANON，回显字符 NL，即使没有设置 ECHO。
ECHOCTL	(不属于 POSIX) 如果同时设置了 ECHO，除了 TAB, NL, START, 和 STOP 之外的 ASCII 控制信号被回显为 ^X, 这里 X 是比控制信号大 0x40 的 ASCII 码。例如，字符 0x08 (BS) 被回显为 ^H。
ECHOPRT	(不属于 POSIX) 如果同时设置了 ICANON 和 IECHO，字符在删除的同时被打印。
ECHOKE	(不属于 POSIX) 如果同时设置了 ICANON，回显 KILL 时将删除一行中的每个字符，如同指定了 ECHOE 和 ECHOPRT 一样。
DEFECHO	(不属于 POSIX) 只在一个进程读的时候回显。
FLUSHO	(不属于 POSIX; Linux 下不被支持) 输出被刷新。这个标志可以通过键入字符 DISCARD 来开关。
NOFLSH	禁止在产生 SIGINT, SIGQUIT 和 SIGSUSP 信号时刷新输入和输出队列。
PENDIN	(不属于 POSIX; Linux 下不被支持) 在读入下一个字符时，输入队列中所有字符被重新输出。(bash 用它来处理 typeahead)
TOSTOP	向试图写控制终端的后台进程组发送 SIGTTOU 信号。
IEXTEN	启用实现自定义的输入处理。这个标志必须与 ICANON 同时使用，才能解释特殊字符 EOL2，LNEXT，REPRINT 和 WERASE，IUCLC 标志才有效。

c_cc 数组定义了特殊的控制字符。符号下标 (初始值) 和意义为：

标志	说明
VINTR	(003, ETX, Ctrl-C, or also 0177, DEL, rubout) 中断字符。发出 SIGINT 信号。当设置 ISIG 时可被识别，不再作为输入传递。
VQUIT	(034, FS, Ctrl-) 退出字符。发出 SIGQUIT 信号。当设置 ISIG 时可被识别，不再作为输入传递。
VERASE	(0177, DEL, rubout, or 010, BS, Ctrl-H, or also #) 删除字符。删除上一个还没有删掉的字符，但不删除上一个 EOF 或行首。当设置 ICANON 时可被识别，不再作为输入传递。
VKILL	(025, NAK, Ctrl-U, or Ctrl-X, or also @) 终止字符。删除自上一个 EOF 或行首以来的输入。当设置 ICANON 时可被识别，不再作为输入传递。
VEOF	(004, EOT, Ctrl-D) 文件尾字符。更精确地说，这个字符使得 tty 缓冲中的内容被送到等待输入的用户程序中，而不必等到 EOL。如果它是一行的第一个字符，那么用户程序的 read() 将返回 0，指示读到了 EOF。当设置 ICANON 时可被识别，不再作为输入传递。
VMIN	非 canonical 模式读的最小字符数。
VEOL	(0, NUL) 附加的行尾字符。当设置 ICANON 时可被识别。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

标志	说明
VTIME	非 canonical 模式读时的延时，以十分之一秒为单位。
VEOL2	(not in POSIX; 0, NUL) 另一个行尾字符。当设置 ICANON 时可被识别。
VSTART	(021, DC1, Ctrl-Q) 开始字符。重新开始被 Stop 字符中止的输出。当设置 IXON 时可被识别，不再作为输入传递。
VSTOP	(023, DC3, Ctrl-S) 停止字符。停止输出，直到键入 Start 字符。当设置 IXON 时可被识别，不再作为输入传递。
VSUSP	(032, SUB, Ctrl-Z) 挂起字符。发送 SIGTSTP 信号。当设置 ISIG 时可被识别，不再作为输入传递。
VLNEXT	(not in POSIX; 026, SYN, Ctrl-V) 字面上的下一个。引用下一个输入字符，取消它的任何特殊含义。当设置 IEXTEN 时可被识别，不再作为输入传递。
VWERASE	(not in POSIX; 027, ETB, Ctrl-W) 删除词。当设置 ICANON 和 IEXTEN 时可被识别，不再作为输入传递。
VREPRINT	(not in POSIX; 022, DC2, Ctrl-R) 重新输出未读的字符。当设置 ICANON 和 IEXTEN 时可被识别，不再作为输入传递。

3.3.1 tcgetattr

- 作用：获取串口设备的属性。
- 参数：
  - fd，串口设备的文件描述符。
  - termios_p，用于保存串口属性。
- 返回：
  - 成功，返回 0。
  - 失败，返回-1，errnor 给出具体错误码。

3.3.2 tcsetattr

- 作用：设置串口设备的属性。
- 参数：
  - fd，串口设备的文件描述符。
  - optional_actions，本次设置什么时候生效。
  - termios_p，指向要设置的属性结构。
- 返回：
  - 成功，返回 0。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

— 失败，返回-1，errnor 给出具体错误码

注意

其中，*optional_actions* 的取值有：
TCSANOW：会立即生效。
TCSADRAIN：当前的输出数据完成传输后生效，适用于修改了输出相关的参数。
TCSAFLUSH：当前的输出数据完成传输，如果输入有数据可读但没有读就会被丢弃。

3.3.3 cfgetispeed

- 作用：返回串口属性中的输入波特率。
- 参数：
  — termios_p，指向保存有串口属性的结构。
- 返回：
  — 成功，返回波特率，取值是一组宏，定义在 terminos.h。
  — 失败，返回-1，errnor 给出具体错误码。

注意

波特率定义如下所示：

```
#define B0 0000000
#define B50 0000001
#define B75 0000002
#define B110 0000003
#define B134 0000004
#define B150 0000005
#define B200 0000006
#define B300 0000007
#define B600 0000010
#define B1200 0000011
#define B1800 0000012
#define B2400 0000013
#define B4800 0000014
#define B9600 0000015
#define B19200 0000016
#define B38400 0000017
#define B57600 0010001
#define B115200 0010002
#define B230400 0010003
#define B460800 0010004
#define B500000 0010005
#define B576000 0010006
#define B921600 0010007
#define B1000000 0010010
#define B1152000 0010011
#define B1500000 0010012
#define B2000000 0010013
#define B2500000 0010014
#define B3000000 0010015
#define B3500000 0010016
#define B4000000 0010017
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.3.4 cfgetospeed

- 作用：返回串口属性中的输出波特率。
- 参数：
  - termios_p，指向保存有串口属性的结构。
- 返回：
  - 成功，返回波特率，取值是一组宏，定义在 terminos.h。
  - 失败，返回-1，errnor 给出具体错误码。

3.3.5 cfsetispeed

- 作用：设置输入波特率到属性结构中。
- 参数：
  - termios_p，指向保存有串口属性的结构。
  - speed，波特率。
- 返回：
  - 成功，返回 0。
  - 失败，返回-1，errnor 给出具体错误码。

3.3.6 cfsetospeed

- 作用：设置输出波特率到属性结构中。
- 参数：
  - termios_p，指向保存有串口属性的结构。
  - speed，波特率。
- 返回：
  - 成功，返回 0。
  - 失败，返回-1，errnor 给出具体错误码

3.3.7 cfsetspeed

- 作用：同时设置输入和输出波特率到属性结构中。
- 参数：
  - termios_p，指向保存有串口属性的结构。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

  - speed，波特率。

- 返回：
  - 成功，返回 0。
  - 失败，返回-1，errnor 给出具体错误码

3.3.8 tcflush

- 作用：清空输出缓冲区、或输入缓冲区的数据，具体取决于参数 queue_selector。
- 参数：
  - fd，串口设备的文件描述符。
  - queue_selector，清空数据的操作。
- 返回：
  - 成功，返回 0。
  - 失败，返回-1，errnor 给出具体错误码。

注意

参数 *queue_selector* 的取值有三个：
TCIFLUSH：清空输入缓冲区的数据。
TCOFLUSH：清空输出缓冲区的数据。
TCIOFLUSH：同时清空输入/输出缓冲区的数据。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 模块使用范例

此 demo 程序是打开一个串口设备，然后侦听这个设备，如果有数据可读就读出来并打印。设备名称、侦听的循环次数都可以由参数指定。

```
#include     <stdio.h>       /*标准输入输出定义*/
#include     <stdlib.h>      /*标准函数库定义*/
#include     <unistd.h>      /*Unix 标准函数定义*/
#include     <sys/types.h>
#include     <sys/stat.h>
#include     <fcntl.h>       /*文件控制定义*/
#include     <termios.h>     /*PPSIX 终端控制定义*/
#include     <errno.h>       /*错误号定义*/
#include <string.h>

enum parameter_type {
    PT_PROGRAM_NAME = 0,
    PT_DEV_NAME,
    PT_CYCLE,

    PT_NUM
};

#define DBG(string, args...) \
    do { \
        printf("%s, %s()%u---", __FILE__, __FUNCTION__, __LINE__); \
        printf(string, ##args); \
        printf("\n"); \
    } while (0)

void usage(void)
{
    printf("You should input as: \n");
    printf("\t select_test [/dev/name] [Cycle Cnt]\n");
}

int OpenDev(char *name)
{
    int fd = open(name, O_RDWR );          //| O_NOCTTY | O_NDELAY
    if (-1 == fd)
        DBG("Can't Open(%s)!", name);

    return fd;
}

/**
*@brief  设置串口通信速率
*@param  fd     类型 int  打开串口的文件句柄
*@param  speed  类型 int  串口速度
*@return  void
*/
void set_speed(int fd, int speed){
    int   i;
```

```
49      int  status;
50      struct termios  Opt = {0};
51      int speed_arr[] = { B38400, B19200, B9600, B4800, B2400, B1200, B300,
52                B38400, B19200, B9600, B4800, B2400, B1200, B300, };
53      int name_arr[] = {38400,  19200,  9600,  4800,  2400,  1200,  300, 38400,
54                19200,  9600, 4800, 2400, 1200,  300, };
55
56      tcgetattr(fd, &Opt);
57
58      for ( i= 0;  i < sizeof(speed_arr) / sizeof(int);  i++) {
59          if  (speed == name_arr[i])
60              break;
61      }
62
63      tcflush(fd, TCIOFLUSH);
64      cfsetispeed(&Opt, speed_arr[i]);
65      cfsetospeed(&Opt, speed_arr[i]);
66
67      Opt.c_lflag  &= ~(ICANON | ECHO | ECHOE | ISIG);  /*Input*/
68      Opt.c_oflag  &= ~OPOST;   /*Output*/
69
70      status = tcsetattr(fd, TCSANOW, &Opt);
71      if  (status != 0) {
72          DBG("tcsetattr fd");
73          return;
74      }
75      tcflush(fd, TCIOFLUSH);
76  }
77
78  /**
79  *@brief   设置串口数据位，停止位和效验位
80  *@param  fd      类型  int  打开的串口文件句柄
81  *@param  databits 类型  int 数据位   取值 为 7 或者8
82  *@param  stopbits 类型  int 停止位   取值为 1 或者2
83  *@param  parity  类型  int  效验类型 取值为N,E,O,,S
84  */
85  int set_Parity(int fd,int databits,int stopbits,int parity)
86  {
87      struct termios options;
88
89      if  ( tcgetattr( fd,&options)  !=  0) {
90          perror("SetupSerial 1");
91          return -1;
92      }
93      options.c_cflag &= ~CSIZE;
94
95      switch (databits) /*设置数据位数*/
96      {
97      case 7:
98          options.c_cflag |= CS7;
99          break;
100     case 8:
101         options.c_cflag |= CS8;
102         break;
103     default:
104         fprintf(stderr,"Unsupported data size\n");
105         return -1;
106     }
107
108     switch (parity)
```

```
109     {
110     case 'n':
111     case 'N':
112         options.c_cflag &= ~PARENB;   /* Clear parity enable */
113         options.c_iflag &= ~INPCK;     /* Enable parity checking */
114         break;
115     case 'o':
116     case 'O':
117         options.c_cflag |= (PARODD | PARENB); /* 设置为奇效验*/
118         options.c_iflag |= INPCK;             /* Disable parity checking */
119         break;
120     case 'e':
121     case 'E':
122         options.c_cflag |= PARENB;     /* Enable parity */
123         options.c_cflag &= ~PARODD;   /* 转换为偶效验*/
124         options.c_iflag |= INPCK;       /* Disable parity checking */
125         break;
126     case 'S':
127     case 's':  /*as no parity*/
128         options.c_cflag &= ~PARENB;
129         options.c_cflag &= ~CSTOPB;break;
130     default:
131         fprintf(stderr,"Unsupported parity\n");
132         return -1;
133     }
134
135     /* 设置停止位*/
136     switch (stopbits)
137     {
138         case 1:
139             options.c_cflag &= ~CSTOPB;
140             break;
141         case 2:
142             options.c_cflag |= CSTOPB;
143            break;
144         default:
145              fprintf(stderr,"Unsupported stop bits\n");
146             return -1;
147     }
148
149     /* Set input parity option */
150     if (parity != 'n')
151         options.c_iflag |= INPCK;
152     tcflush(fd,TCIFLUSH);
153     options.c_cc[VTIME] = 150; /* 设置超时15 seconds*/
154     options.c_cc[VMIN] = 0; /* Update the options and do it NOW */
155     if (tcsetattr(fd,TCSANOW,&options) != 0)
156     {
157         perror("SetupSerial 3");
158         return -1;
159     }
160     return 0;
161 }
162
163 void str_print(char *buf, int len)
164 {
165     int i;
166
167     for (i=0; i<len; i++) {
168         if (i%10 == 0)
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
169             printf("\n");
170
171         printf("0x%02x ", buf[i]);
172     }
173     printf("\n");
174 }
175
176 int main(int argc, char **argv)
177 {
178     int i = 0;
179     int fd = 0;
180     int cnt = 0;
181     char buf[256];
182
183     int ret;
184     fd_set rd_fdset;
185     struct timeval dly_tm;      // delay time in select()
186
187     if (argc != PT_NUM) {
188         usage();
189         return -1;
190     }
191
192     sscanf(argv[PT_CYCLE], "%d", &cnt);
193     if (cnt == 0)
194         cnt = 0xFFFF;
195
196     fd = OpenDev(argv[PT_DEV_NAME]);
197     if (fd < 0)
198         return -1;
199
200     set_speed(fd,19200);
201     if (set_Parity(fd,8,1,'N') == -1)  {
202         printf("Set Parity Error\n");
203         exit (0);
204     }
205
206     printf("Select(%s), Cnt %d. \n", argv[PT_DEV_NAME], cnt);
207     while (i<cnt) {
208         FD_ZERO(&rd_fdset);
209         FD_SET(fd, &rd_fdset);
210
211         dly_tm.tv_sec  = 5;
212         dly_tm.tv_usec = 0;
213         memset(buf, 0, 256);
214
215         ret = select(fd+1, &rd_fdset, NULL, NULL, &dly_tm);
216 //      DBG("select() return %d, fd = %d", ret, fd);
217         if (ret == 0)
218             continue;
219
220         if (ret < 0) {
221             printf("select(%s) return %d. [%d]: %s \n", argv[PT_DEV_NAME], ret, errno,
    strerror(errno));
222             continue;
223         }
224
225         i++;
226         ret = read(fd, buf, 256);
227         printf("Cnt%d: read(%s) return %d.\n", i, argv[PT_DEV_NAME], ret);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
        str_print(buf, ret);
    }

    close(fd);
    return 0;
}
```

广州星翼电子科技有限公司zdyzlinux

5 调试方法

- 通过 debugfs 使用命令打开调试开关。

注：内核需打开CONFIG_DYNAMIC_DEBUG宏定义。

```
1.挂载debugfs。
mount -t debugfs none /sys/kernel/debug
2.打开uart模块所有打印。
echo "module sunxi_uart +p" > /sys/kernel/debug/dynamic_debug/control
3.打开指定文件的所有打印。
echo "file sunxi-uart.c +p" > /sys/kernel/debug/dynamic_debug/control
4.打开指定文件指定行的打印。
echo "file sunxi-uart.c line 615 +p" > /sys/kernel/debug/dynamic_debug/control
5.打开指定函数名的打印。
echo "func sw_uart_set_termios +p" > /sys/kernel/debug/dynamic_debug/control
6.关闭打印。
把上面相应命令中的+p 修改为-p 即可。
更多信息可参考linux 内核文档：linux-3.10/Documentation/dynamic-debug-howto.txt。
```

5.1 调试工具

5.2 调试节点

5.2.1 /sys/devices/platform/soc*/*uart*/dev_info

从该节点可以看到 UART 端口的一些硬件资源信息。以内核为 linux6.6 的 a733 下的uart0为例：

```
# cat /sys/devices/platform/soc@3000000/2b00000.uart/dev_info
id      = 0
name    = uart0
irq     = 247
io_num = 2
port->mapbase = 0x0000000005000000
port->membase = 0xffffff800b005000
port->iobase  = 0x00000000
pdata->regulator     = 0x          (null)
pdata->regulator_id =
```

- id：uart编号。
- name：uart名称。
- irq：uart中断号。
- io_num：表明 uart0 目前被配置为为 2 线模式。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.2.2 /sys/devices/platform/soc*/*uart*/status

此节点可以打印出 UART 端口的一些运行状态信息，包括控制器的各寄存器值。以内核为 linux6.6 的 a733 下的`uart0`为例：

```
# cat /sys/devices/platform/soc@3000000/2b00000.uart/status
uartclk = 24000000
The Uart controller register[Base: 0xffffff800b005000]:
[RTX] 0x00 = 0x0000000d, [IER] 0x04 = 0x00000005, [FCR] 0x08 = 0x000000c1
[LCR] 0x0c = 0x00000013, [MCR] 0x10 = 0x00000003, [LSR] 0x14 = 0x00000060
[MSR] 0x18 = 0x00000000, [SCH] 0x1c = 0x00000000, [USR] 0x7c = 0x00000006
[TFL] 0x80 = 0x00000000, [RFL] 0x84 = 0x00000000, [HALT] 0xa4 = 0x00000002
```

5.2.3 /sys/devices/platform/soc*/*uart*/ctrl_info

此节点可以打印出软件中保存的一些控制信息，如当前 UART 端口的寄存器值、收发数据的统计等。以内核为 linux6.6 的 a733 下的`uart0`为例：

```
# cat /sys/devices/platform/soc@3000000/2b00000.uart/ctrl_info
 ier  : 0x05
 lcr  : 0x13
 mcr  : 0x03
 fcr  : 0xb1
 dll  : 0x0d
 dlh  : 0x00
 last baud : 115384 (dl = 13)

TxRx Statistics:
 tx     : 61123
 rx     : 351
 parity : 0
 frame  : 0
 overrun: 0
```

重要信息如下：

- tx：当前发出的数据总量。
- rx：接收到的数据总量。
- frame：frame 错误的次数。
- overrun：overrun 的次数，代表当前 FIFO 缓冲区溢出。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 常见需求及解决方案

6.1 控制台相关问题

6.1.1 如何切换控制台

1. dts 确保想作为控制台的uart已经使能，status节点设置为okay，如 uart1。

```
uart1: uart@05000400 {
  compatible = "allwinner,sun50i-uart";
  device_type = "uart1";
  reg = <0x0 0x05000400 0x0 0x400>;
  interrupts = <GIC_SPI 1 IRQ_TYPE_LEVEL_HIGH>;
  clocks = <&clk_uart1>;
  pinctrl-names = "default", "sleep";
  pinctrl-0 = <&uart1_pins_a>;
  pinctrl-1 = <&uart1_pins_b>;
  uart1_port = <1>;
  uart1_type = <4>;
  status = "okay"; /* 确保该uart已经使能 */
};
```

2. 修改方案使用的sys_config.fex文件，按照实际情况填写 Pin 脚信息，如下所示：

```
-[uart_para]
-uart_debug_port = 0
-uart_debug_tx   = port:PB09<2><1><default><default>
-uart_debug_rx   = port:PB10<2><1><default><default>
+[uart_para]
+uart_debug_port = 1
+uart_debug_tx   = port:PG06<2><1><default><default>
+uart_debug_rx   = port:PG07<2><1><default><default>
```

3. 修改方案使用的env*.cfg文件，只修改 tty 序号，剩余部分不变动，一般 tty 序号就是 uart 序号，如下所示：

```
-console=ttyS0,115200
+console=ttyS1,115200

或是

-console=ttyAS0,115200
+console=ttyAS1,115200
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.1.2 如何切换控制台波特率

分两个步骤

- 首先保证作为控制台的uart口，在dtsi中的时钟配置，可以支持想要配置的波特率，时钟节点配置方法详见“波特率配置说明”
- 第二步是在时钟节点正确的基础上，修改配置文件，保证启动过程按预设波特率打印 log，配置方法如下所示

根据 SDK 发布时间不同，有两种配置方法

- 部分平台可在sys_config.fex文件中快速切换控制台波特率，且默认控制台波特率为1.5 MHz，平台列表与配置方法详见章节“部分平台控制台1.5 MHz波特率相关问题”
- 剩余大部分平台默认波特率为 115200 bps，且需要以下几处才能修改全阶段的控制台波特率
- spl阶段

  在SDK\brandy\brandy-2.0\spl\include\arch\uart.h中将默认波特率改为想要设置的波特率，若是spl未开源则需要联系spl负责人重新给出spl镜像

```
- #define   UART_BAUD    115200
+ #define   UART_BAUD    XXX
```

- uboot 阶段

  在当前平台所用的defconfig文件中添加波特率配置项，路径一般为SDK\brandy\brandy-2.0\u-boot-2018\configs\sunXXX_defconfig

```
+ CONFIG_BAUDRATE=XXX
```

- 内核阶段

  修改方案使用的env*.cfg文件，只修改波特率部分，剩余部分不变动

```
- console=ttyAS0,115200
+ console=ttyAS0,XXX
```

6.1.3 部分平台控制台 1.5 MHz 波特率相关问题

已在部分平台上将控制台波特率调整至 1.5 MHz，在使用这些平台时需要严格遵守以下注意事项。

6.1.3.1 原因

过往项目的串口打印波特率均为 115200 bps，打印效率低，大约 10 个字符耗时 1ms，存在诸多性能问题。故在 A537/A333 上开始，推行 1.5 MHz 波特率，后续使用需要注意波特率的切换，否则波特率不匹配会出现串口卡住或乱码。

当前各平台串口波特率配置如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

芯片	波特率
A733	115200 bps
A523	115200 bps
A527	115200 bps
A537	1500000 bps
A333	1500000 bps

6.1.3.2 硬件要求

使用 1.5 MHz 波特率需要注意如下事项。

1. **尽量选择 FT232 的串口线，其次是 CH340，若未使用要求用线，可能会出现乱码、丢数、行数混乱的情况，以下为数据错位示例；**

```
[2025-03-18 23:09:38]  |1200 100%|  |1200 0%|  |1536 100%|  |1536 100%| 1512 100%|  79344   790   79982   200   788      792   14796
[2025-03-18 23:09:39]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  344   78880   79460  00   78590      792  4797
[2025-03-18 23:09:39]  |1200 100%|  |1200 0%|  |1536 100%|  |1536 100%||1512 100%|  79692   78822   804   200   79112      792   147
[2025-03-18 23:09:40]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  040   78996   79808  00   78474      792   147
[2025-03-18 23:09:40]----------------------------------------------------------a537 CPU[x]req:MHZ>--<Usage:%>----------------------------------------------------------
[2025-03-18 23:09:40]      CPU0        CPU1       CPU2        CPU3       PU4    TempL    TempB    TempM   GPU  GPUTp   DDR  Times
[2025-03-18 23:09:40]  |1200 100%||1200 100%|  |1536 100%||1536 100%|  |1512 100%|  79982   790   79750   696   78938    792   14800
[2025-03-18 23:09:41]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  460   79228   80330  00   78300      792  4801
[2025-03-18 23:09:41]  |1200 100%|  |1200 0%|  |1536 100%|  |1536 10|  |1512 100%|  79924 78938   79692   200 78880      792   1480
[2025-03-18 23:09:42]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  79170   79170   800   696   78358      2   14803
[2025-03-18 23:09:42]  |1200 100%|  |1200 0%|  |1536 100%|  |153600%|  |1512 100%|  792   79692   80272   2   78706      792   148
[2025-03-18 23:09:43][11592.701726][  T472] healthd: battery none chg=au
[2025-03-18 23:09:43]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  112   79518   80562  00   78822      792  4805
[2025-03-18 23:09:43]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  852   78358   80330  96   78532      792  4806
[2025-03-18 23:09:44]  |1200 100%|  |1200 0%|  |1536 100%|  |1536 0%|  |1512 100%|  790   79112   80330   20  79576      792   147
[2025-03-18 23:09:44]  |1200 100%|  |1200 0%|  |1536 100%|  |1536 100% |1512 100%|  80272   79576   800   200   80446      792   148
[2025-03-18 23:09:45]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  692   78996   80562  96   78590      792   109
[2025-03-18 23:09:45]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  866   80272   80910  00   78764      792  4810
[2025-03-18 23:09:46]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  808   78880   80852  00   79344      792  4811
[2025-03-18 23:09:46]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  750   79286   80388  96   79112      792  4812
[2025-03-18 23:09:47]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  79576   78996   800   200   78474      2   14813
[2025-03-18 23:09:48]  |1200 100%|  |1200 0%|  |1536 100%|  |153100%|  |1512 100%|  79866 79576   80852   200  9054      792   14814
[2025-03-18 23:09:48]          |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  460   79170   80272  00   79112      792  4815
[2025-03-18 23:09:48]----------------------------------------------------------a537 CPU[x]<Freq:MHZ>--<age:%>----------------------------------------------------------
[2025-03-18 23:09:48]    CPU0        CPU1       CPU2        CPU3       CPU4    TempL    TempB    TempM   GPU PUTemp   DDR  Times
[2025-03-18 23:09:49]|1200 100%|  |1200 10|  |1536 100%|  |153600%|  |1512 100%|  802   79054   79924   20  78938      792   146
[2025-03-18 23:09:49]  |1200 100%|  |1200 0%|  |1536 100%|  |1536 1%|  |1512 100%|  7975   79170   80388   200 79460      792   148
[2025-03-18 23:09:50]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  692   79054   80446  00   79054      792  4818
[2025-03-18 23:09:50]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  460   79112   80620  00   78938      792  4819
[2025-03-18 23:09:51]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  504   78648   79286  00   78822      792  4820
[2025-03-18 23:09:51]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  692   78938   80272  96   78590      792   148
[2025-03-18 23:09:52]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  098   79402   81316  00   78938      792  4822
[2025-03-18 23:09:52]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  228   79518   80330  00   77720      792  4823
[2025-03-18 23:09:53]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  112   79402   79924  96   78300      792  4824
[2025-03-18 23:09:53][11602.947495][  T472] healthd: battery none chg=au
[2025-03-18 23:09:53]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  214   79054   81084  00   78764      792  4825
[2025-03-18 23:09:54]  |1200 100%|  |1200 0%|  |1536 100%|  |1536 100|  |1512 100%|  80040 80156   80910   200 78126      792   1482
[2025-03-18 23:09:54]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  982   78648   80040  00   79170      792   148
[2025-03-18 23:09:55]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  562   79808   79982  00   78474      792  4828
[2025-03-18 23:09:55]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  214   80156   79750  00   79634      792  4829
[2025-03-18 23:09:56]  |1200 100%|  |1200 0%|  |1536 100%|  |1536 100%| 1512 100%|  79808   80040   802   200   78822      792   148
[2025-03-18 23:09:56]  |1200 100%|  |1200 0%|  |1536 100%|  |1536 1%|  |1512 100%|  8015   79344   80446   200 78532      792   148
[2025-03-18 23:09:57]----------------------------------------------------------a537 CPU[x]req:MHZ>--<Usage:%>----------------------------------------------------------
[2025-03-18 23:09:57]      CPU0        CPU1       CPU2        CPU3        CPU   TempL    TempB    TempM   GPU  GPUTp   DDR  Times
[2025-03-18 23:09:57]  |12 100%|  |1200 100%|  536 100%|  |1536 100% |1512 100%|  79402  9402   80446   200   300      792   14832  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  286   79170   80040  00   78358      792  4833
[2025-03-18 23:09:58]  |1200 100%|  |1200 0%|  |1536 100%|  |15 100%|  |1512 100%|  040   79344   80620  00   78300      792  4834
```

图 6-1: 数据错位示例

2. **串口工具设置为 1.5 MHz 波特率，mobaxterm 和 secureCRT 的波特率选项中没有 1.5 MHz 的选项，可手动输入设置 1.5 MHz。**

⚠ 警告

PL2303 的串口线不可用。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.1.3.3 如何切换回 115200 bps 波特率

考虑到终端部分串口线不达要求，存在需要将波特率切回 115200 bps 的需求，而串口波特率切换较为繁琐，故提供下列快速切换方式，便于客户端修改。

1. 修改路径

当前修改波特率只需要修改 sysconfig.fex 中的配置即可，sysconfig.fex 路径如下。

```
longan/device/config/chips/${chip}/configs/${board}/sysconfig.fex
```

⚠ 警告

如果是 GO 平台，如 A333-64bit-go 方案，则使用的 sysconfig.fex 为 sys_config_ddr_3GB.fex。

2. 修改方法

修改 uart_baudrate 的值为 115200 bps，即可将串口输出切换回 115200 bps 波特率，如无该配置，则说明不支持从 sysconfig.fex 中进行配置波特率。

```
[uart_para]
uart_debug_port = 0
uart_debug_tx   = port:PB9<2><1><default><default>
uart_debug_rx   = port:PB10<2><1><default><default>
uart_baudrate   = 1500000
```

⚠ 警告

当前仅 A537 和 A333 平台支持快速切换，后续会支持更多平台。

6.2 关于 uart 波特率

6.2.1 特殊波特率配置方法-手动计算

- 先通过公式确定对apb时钟以及apb时钟源频点的要求，选定合适的时钟源，查询时钟宏名。
- 参考 “计算后修改dtsi中的时钟配置” 章节配置节点。

6.2.1.1 获取支持特定波特率的时钟信息

6.2.1.1.1 apb 时钟

- 获取频点信息。

这个信息会用于挑选apb的时钟源。

apb时钟频点最小值 = 目标波特率 * 16，此数值的整数倍频点均可以支持此波特率。

apb作为总线时钟有范围限制，详情查看ccmu spec的“总线时钟配置”章节，一般为100M或160M，在符合总线要求的情况下取最大值。

示例：想支持6.25M波特率，apb时钟最小值为6.25*16=100M，再结合总线频率限制，确认apb时钟取值 100M。

- 获取apb时钟宏名信息。

apb只是一个代称，具体的时钟宏名可以在时钟代码中查看，宏名信息需要填写至dtsi文件。

（1）首先查看 dtsi 文件获取 uart 时钟宏名。

dtsi文件的路径为SDK/bsp/configs/linux-XX/sunXXiwXX.dtsi，在此文件中找到想要修改的uart节点的时钟信息。

```
uart2: uart@2B02000 {
    ...
    clocks = <&ccu CLK_BUS_UART2>;
    ...
    }
```

（2）在clk驱动文件SDK/bsp/drivers/clk/sunxi-ng/ccu-sunXXiwX.c，中搜索uart时钟宏名：CLK_BUS_UART2，可以从下列代码中看出此平台uart时钟在代码中的命名为bus_uart2_clk。

```
        [CLK_BUS_UART2]          = &bus_uart2_clk.common.hw,
```

在文件中继续搜索bus_uart2_clk，可以找到时钟定义的地方，其中apb-uart部分为uart的时钟源。

```
static SUNXI_CCU_GATE(bus_uart2_clk, "bus-uart1",
        "apb-uart",
        0x0E04, BIT(0), 0);
```

（3）继续在文件中搜索apb-uart，找到定义 apb 时钟的地方，继续搜索apb_uart_clk，找到 apb 的宏名CLK_APB_UART。

```
static const char * const apb_uart_parents[] = { "sys24 MHz", "osc32k", "iosc", "pll-peri0
    -600m", "pll-peri0-480m" };

SUNXI_CCU_M_WITH_MUX(apb_uart_clk, "apb-uart", apb_uart_parents,
        0x0538, 0, 5, 24, 3, CLK_SET_RATE_PARENT);

...
        [CLK_APB_UART]           = &apb_uart_clk.common.hw,
...
```

6.2.1.1.2 apb 时钟源

- 获取频点信息。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

apb时钟源经过分频之后得到apb时钟，因此选取时钟源时需要能够整除上一步算出的apb时钟频点，有两种情况：

（1）能够找到可以整除apb频点的时钟源。

示例：上一步已经确定6.25M的波特率需要apb时钟频点为100M，再结合查找 apb 时钟宏名时看到的时钟源信息，在五个时钟源中，只有pll-peri0-600m能够满足整数apb目标频点100M，因此选定此时钟作为apb时钟源。

```
static const char * const apb_uart_parents[] = { "sys24 MHz", "osc32k", "iosc", "pll-peri0
    -600m", "pll-peri0-480m" };

SUNXI_CCU_M_WITH_MUX(apb_uart_clk, "apb-uart", apb_uart_parents,
        0x0538, 0, 5, 24, 3, CLK_SET_RATE_PARENT);
```

（2）无法找到可以整除apb频点的时钟源。

此时可以尝试选最大频点的apb时钟源600M，此时apb考虑总线频点限制后为100M，将100M的apb代入“计算误码率”章节，若是误码率过大（建议小于 1.725%），则更换次大的apb时钟源480M，此时apb考虑总线频点限制后为96M，将96M的apb代入“计算误码率”章节，若是误码率依然过大，则表示此平台无法支持目标波特率，若是误码率较小，则可以使用。

- 获取apb时钟源宏名信息。。

上一步已经找到apb时钟定义的地方，其中apb_uart_parents部分为apb所有可选的时钟源，例如当想选择pll-peri0-600m作为 apb 时钟的时钟源时，继续搜索pll-peri0-600m, 可以继续找到时钟定义处和宏定位的位置，宏名信息需要填写至dtsi文件。

```
static CLK_FIXED_FACTOR(pll_peri0_600m_clk, "pll-peri0-600m", "pll-peri0-2x", 2, 1, 0);
...
        [PERI_600M_BUS]     = &pll_peri0_600m_clk.hw,
```

6.2.1.2 计算后修改 dtsi 中的时钟配置

在做完上章节的查询和计算工作之后，我们得知在想配置6.25M波特率时，时钟的信息如下：

- apb时钟宏名：CLK_APB_UART，apb频点要求：100M的整数倍。
- 符合apb频点要求的apb时钟源宏名：PERI_600M_BUS，此时钟从名称可以看出频点是600M。

将以上信息填入SDK/bsp/configs/linux-XX/sunXXiwXX.dtsi中。

以下代码为例，讲解上一章节计算和查找到的信息如何填入dtsi文件，一共需要修改两个节点。

```
            device_type = "uart2";
            reg = <0x0 0x05000800 0x0 0x400>;
            interrupts = <GIC_SPI 2 IRQ_TYPE_LEVEL_HIGH>;
-                       clocks = <&ccu CLK_BUS_UART2>;
                        //将dtsi文件中原有的clocks节点部分继续放在新的clocks节点的第一个成员即可
+                       clocks = <&ccu CLK_BUS_UART2>,
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
                                        //将apb时钟宏名填入clocks节点的第二个成员
+                                        <&ccu CLK_APB_UART>,
                                        //将apb时钟源宏名填入clocks节点的第三个成员
+                                        <&ccu PERI_600M_BUS>;
                                //将apb时钟频点填入clock-frequency节点
+                               clock-frequency = <100000000>;
                pinctrl-names = "default", "sleep";
```

- 重要注意事项！！！
  - 由于所有uart时钟都来自于同一个apb时钟，因此dtsi的修改需要同步到改到所有的uart节点中，不可以只修改其中一个，否则会影响其他uart口的正常数据传输，例如可能会导致控制台的打印出现乱码。
  - 修改APB总线时钟，可能会影响到使用到APB总线时钟下挂的其他模块，例如twi等，需要根据ccmu spec查询是否会影响到其他模块。

6.2.2 计算误码率

连续传输数据量越大，对误码率要求越高，推荐不超过 1.725%，如果超过，可以考虑使用其他大小接近但误码率更小的波特率。

计算步骤：

- 获取apb时钟频点。
- 如果没有修改过uart节点的clock信息，则可以认为apb时钟为24 MHz。
- 如果已经修改或计划修改，则使用新的apb时钟频点。
- 计算分频值 d，d = apb频点/16/目标波特率。
- 若 d 是整数，则误码率为 0。
- 若 d 带小数部分，则实际使用的是四舍五入后的整数值 d_rounded，此时误码率计算公式为：误码率 = |d - d_rounded| / d × 100%

示例：

```
apb=24 MHz
baudrate=115200
d = 24000000/16/115200 = 13.020833
误码率 = 0.020833 / 13 = 0.16%
```

6.2.3 如何确认波特率设置成功

需要保证上层应用已经调用完波特率配置接口，才能从以下节点中查询到当前实际波特率。

进入uart设置节点下查看设备信息，以下 log 为某平台的具体例子，调试时需要根据平台实际地址更换路径中带地址的部分。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

结果中`last baud : 115384 (dl = 13)`表示此uart目前波特率被设置为 115384。

```
cd /sys/devices/platform/soc@3000000/2500000.uart
cat ctrl_info
 ier  : 0x05
 lcr  : 0x13
 mcr  : 0x03
 fcr  : 0xb1
 dll  : 0x0d
 dlh  : 0x00
 last baud : 115384 (dl = 13)

TxRx Statistics:
 tx     : 20129
 rx     : 146
 parity : 0
 frame  : 0
 overrun: 0
 throttled: 0
```

6.2.4 如何确认修改完 dtsi 后时钟切换成功

在板子上运行以下指令可以查看时钟树。

```
mount -t debugfs none sys/kernel/debug
cat sys/kernel/debug/clk/clk_summary
```

以如下打印为例，可以看到`uartX-bus`时钟挂在`apb-uart`时钟下，`apb-uart`时钟挂在`dcxo24 MHz`时钟下，频率为 24 MHz。

```
 dcxo24 MHz                          29        29        0    24000000          0     0
    50000          Y
    apb-uart                        1         1         0    24000000          0     0  50000
            Y
       uart0-bus                    1         1         0    24000000          0     0  50000
            Y
       uart1-bus                    0         0         0    24000000          0     0  50000
            N
       uart2-bus                    0         0         0    24000000          0     0  50000
            N
       uart3-bus                    0         0         0    24000000          0     0  50000
            Y
```

6.3 如何测试 UART 回环功能

回环测试分为硬件回环与软件回环：

- 若需要使用硬件回环，需要短接 uart 的 tx 与 rx 引脚。
- 若需要使用软件回环，可以在`/sys/devices/platform`路径下找到uart的`loopback`节点，使用如下指令即可配置软件回环：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
echo enable > loopback
```

- 关闭软件回环。

```
echo disable > loopback
```

若需要测试uart回环模式是否正常工作，可向我司获取uarttest文件进行测试。或者参考 “模块使用范例” 章节。

注意：除了"enable"，其他所有字符都表示不开启回环。

6.4 接收数据时如何避免分段

当波特率小于等于 9600 时，uart 接收数据可能会分段。

比如一次性发送数据 “12345678”，但是接收会分多次，第一次接收 “1234”，第二次接收 “5678”。现象类似下图：

图 6-2: 接收数据分段

根本原因：

- 当波特率小于等于 9600 时：

硬件接收到 1 个字节数据时，即触发接收中断。此时 cpu 运行速率远大于 uart 传输速率，因此 cpu 读取完一个字节之后就会立刻上报。此时用户层 read 线程被唤醒，大概率只能读到一个字节数据。

- 大于 9600 波特率时：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

触发接收中断的场景有两种：（1）当接收数据到达 1/2 FIFO 缓冲区深度。

（2）未到达 1/2 FIFO 缓冲区深度，但在一定的时间未接收到数据。

此时 cpu 触发中断的时间会有一定延迟，FIFO 缓冲区中会有多个数据存留。此时用户层 read 线程被唤醒，大概率会完整读到整包数据。

修改方法：

```
@@ -1245,14 +1245,9 @@ static void sw_uart_set_termios(struct uart_port *port, struct
    ktermios *termios
         */
        sw_uart_reset(sw_uport);

-       if (baud <= 9600)
-               sw_uport->fcr = SUNXI_UART_FCR_RXTRG_1CH
-                               | SUNXI_UART_FCR_TXTRG_1_2
-                               | SUNXI_UART_FCR_FIFO_EN;
-       else
-               sw_uport->fcr = SUNXI_UART_FCR_RXTRG_1_2
-                               | SUNXI_UART_FCR_TXTRG_1_2
-                               | SUNXI_UART_FCR_FIFO_EN;
+       sw_uport->fcr = SUNXI_UART_FCR_RXTRG_1_2
+                       | SUNXI_UART_FCR_TXTRG_1_2
+                       | SUNXI_UART_FCR_FIFO_EN;

        serial_out(port, sw_uport->fcr, SUNXI_UART_FCR);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

ALLWINNER、ALLWINNER 全志科技、全志科技、ALLWINNER（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。